CSCI 441 - Software Engineering

SuperAsk.AI: AI-Powered Government Service Assistant

Report Two

Team AO3

Yean Fong, Phallamonunhh Chun, BunKheang Heng, Doungkhe Neang, Heejae Choi, and Sereyprophea Sarin

Professor Hieu Vu

Project URL: https://github.com/Monunhh/Super-ask.git

Table of Contents

SuperAsk.AI: AI-Powered Government Service Assistant..... 1
1. Interaction Diagrams..... 4
2. Class Diagram and Interface Specification..... 10
3. System Architecture..... 45
4. Algorithms and Data Structures..... 53
5. User Interface Design and Implementation..... 59
6. Design of Tests..... 65
Project Management..... 75
References..... 82

Individual Contributions Breakdown

Team Member	Contribution to Full Report Two
Yean Fong	Completed Section 4: Algorithms and Data Structures, including algorithms, data structures, and concurrency; Section 5: User Interface Design and Implementation, covering interface improvements and user effort; and Section 6: Design of Tests, including test cases, test coverage, integration testing strategy, and testing plans. Also contributed to Section 7: Project Management by preparing the Project Coordination and Progress Report, updating the Plan of Work and Gantt chart, reviewing the overall document, assisting team members, and ensuring consistency in content, formatting, and organization.
Doungkhe Neang	Contributed to Section 7: Project Management by coordinating team responsibilities, compiling team members' contributions, and ensuring consistency across different sections of the report.
Sereyprophea Sarin	Contributed to Section 7: Project Management by updating the Plan of Work, organizing project milestones, deadlines, and schedules, and adjusting planned activities based on project progress.

Phallamonunhh Chun	Contributed to Section 1: Interaction Diagrams and Section 2: Class Diagram and Interface Specification, including overview and partial class diagrams, data types, operation signatures, and traceability matrices. Revised Section 3: System Architecture to ensure consistency with the class design and implemented system, including subsystem-to-hardware mapping, connectors and network protocols, global control flow, and hardware requirements.
Bunkheang Heng	Contributed to Section 3(a–c): System Architecture by identifying subsystems, developing the UML package diagram, describing architectural styles, and mapping subsystems to hardware.
Heejae Choi	Contributed to Section 3(d–f): System Architecture by documenting connectors and network protocols and completing the remaining system architecture specifications.

1. Interaction Diagrams

The interaction diagrams below refine the five use cases that were fully dressed in Report One (Sections 3c-iv and 3d): UC-1 AskQuestion, UC-2 RetrieveOfficialSources, UC-3 ViewSourceCitation, UC-7 GetOfficeContactAndLocation, and UC-11 RefreshOfficialContent. Where the system sequence diagrams in Report One treated SuperAsk.AI as a single black box, the sequence diagrams here open that box into design objects and show how responsibilities are assigned to them. The design principles employed are the GRASP principles from the course lecture notes (Section 2.5.1): each assignment decision is marked as a yellow comment bubble directly in the diagram, and summarised again in the caption under each figure.

Design principles employed in assigning responsibilities. Four decisions shape all five diagrams. First, each use case has a single **Controller** (QAController for the citizen-facing flows, RefreshController for the scheduled refresh) that receives system events, so boundary objects such as :ChatUI stay free of business logic. Second, responsibilities are placed with the object that has the information needed to fulfil them (**Information Expert**): ContentStore decides what content is currently in effect, ConversationHistory resolves follow-up references, SourceRegistry knows the registered sources, and OfficeDirectory answers contact questions. Third, work that fits no domain object cleanly is factored into cohesive service classes (**Pure Fabrication**, **High Cohesion**): LanguageService, RetrievalService, AnswerVerifier, CitationPresenter. Fourth, points where change is expected are wrapped behind stable interfaces (**Protected Variations**, **Indirection**, **Polymorphism**): AIModelGateway hides the external AI model vendor, SearchIndex hides the search method, and SourceCrawler hides the source platform, which is what lets REQ-29/REQ-30 hold without redesigning the core.

UC-1: AskQuestion

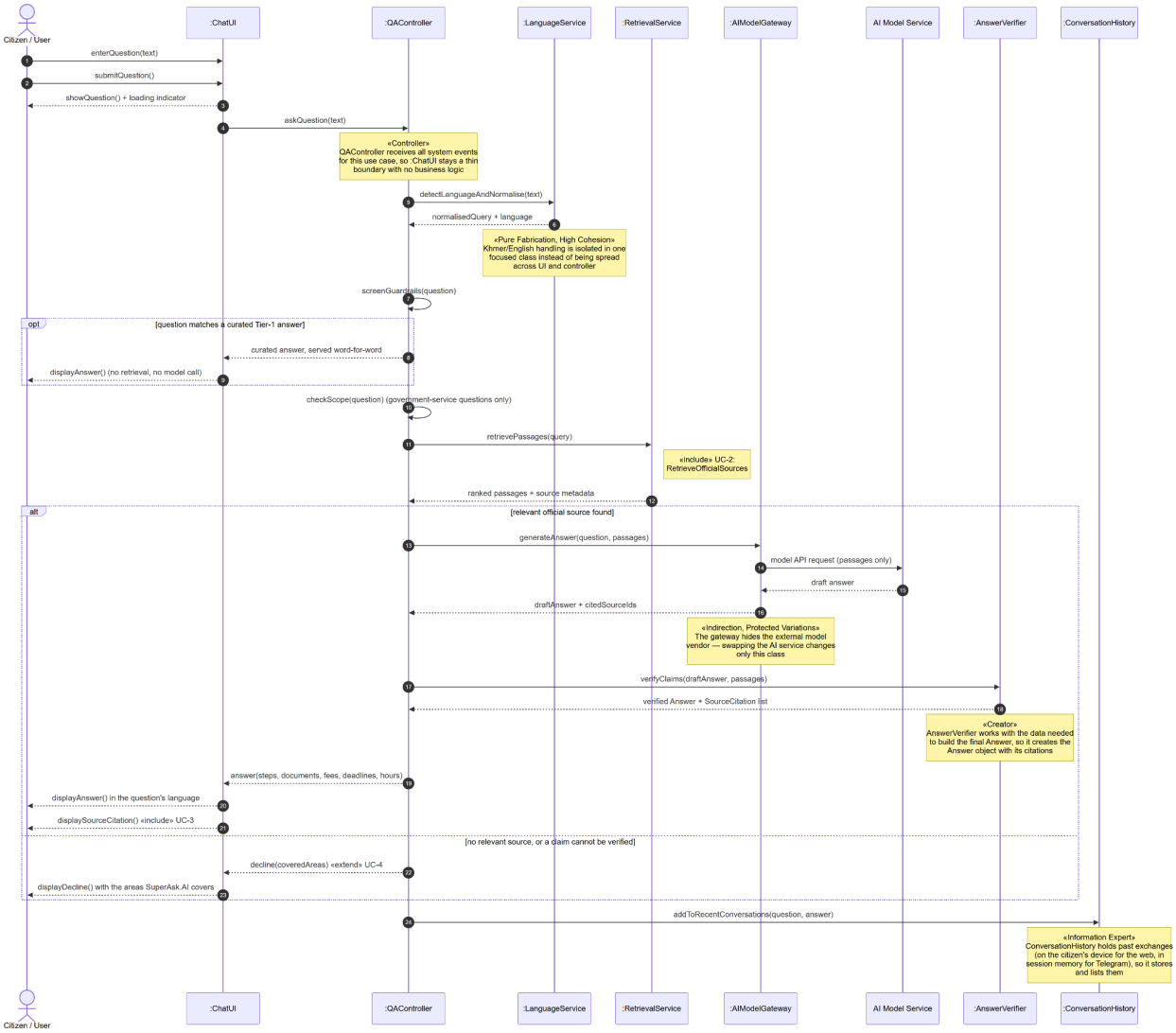


Figure 1: Sequence diagram for UC-1: AskQuestion.

Design principles: the responsibility for handling the user's events is assigned to QAController (**Controller**), which keeps :ChatUI a thin boundary object with no business logic (**Low Coupling**). Khmer/English handling and claim verification are factored into LanguageService and AnswerVerifier (**Pure Fabrication**, **High Cohesion**). AIModelGateway isolates the external AI Model Service behind one interface, so swapping the model vendor changes only that class (**Indirection**, **Protected Variations**). AnswerVerifier assembles the final Answer with its citations because it already works with all the data the Answer needs (**Creator**), and ConversationHistory stores the exchange because it owns the conversation data (**Information Expert**).

UC-2: RetrieveOfficialSources

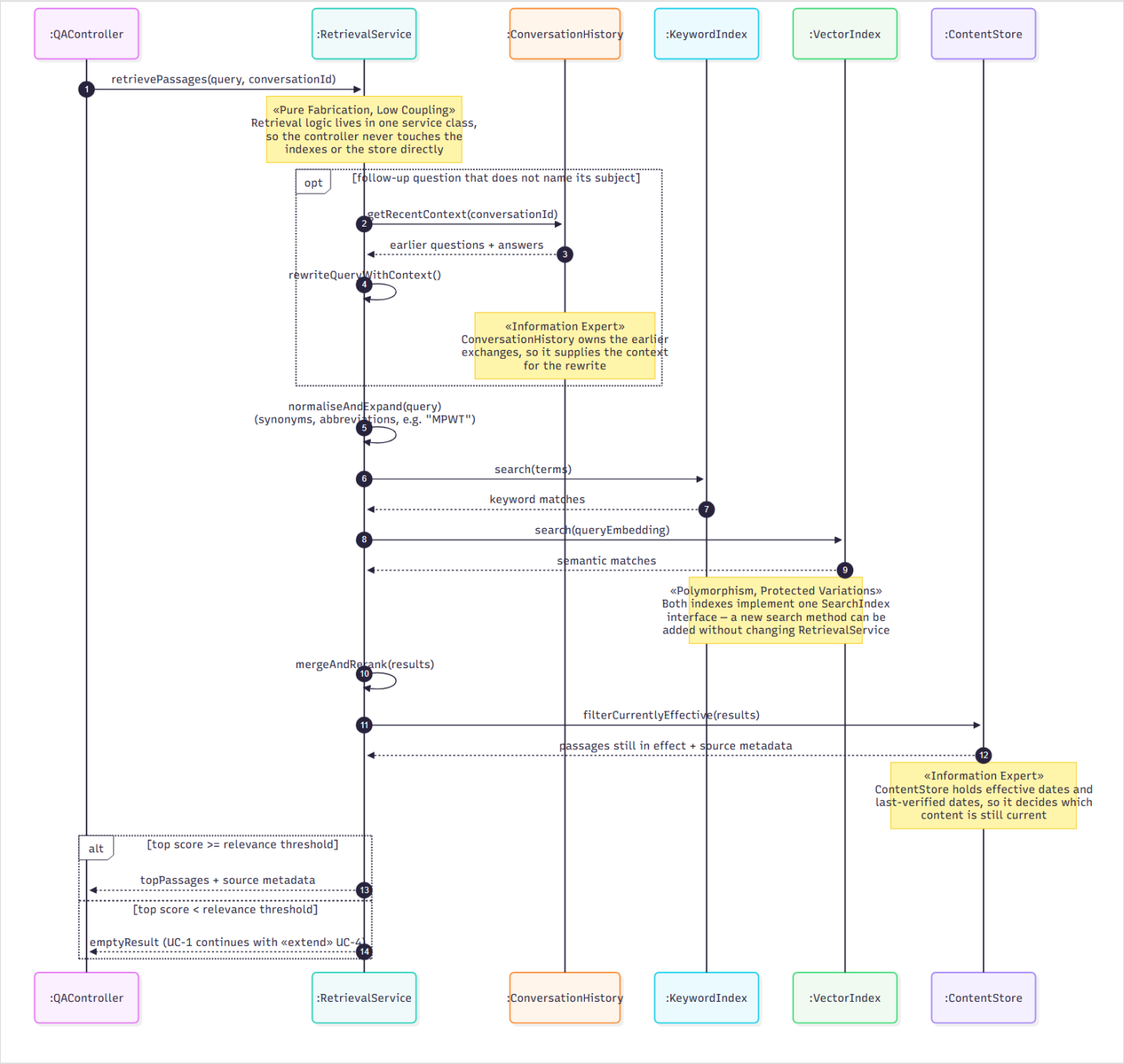


Figure 2: Sequence diagram for UC-2: RetrieveOfficialSources.

Design principles: RetrievalService is a **Pure Fabrication** that keeps all retrieval logic in one cohesive class and shields QAController from the indexes and the content store (**Low Coupling**, **Indirection**). KeywordIndex and VectorIndex implement a single SearchIndex interface (**Polymorphism**), so a new search method can be added without touching RetrievalService

(**Protected Variations**). ConversationHistory supplies the context for rewriting follow-up questions, and ContentStore decides which passages are still in effect, because each owns exactly that data (**Information Expert**).

UC-3: ViewSourceCitation

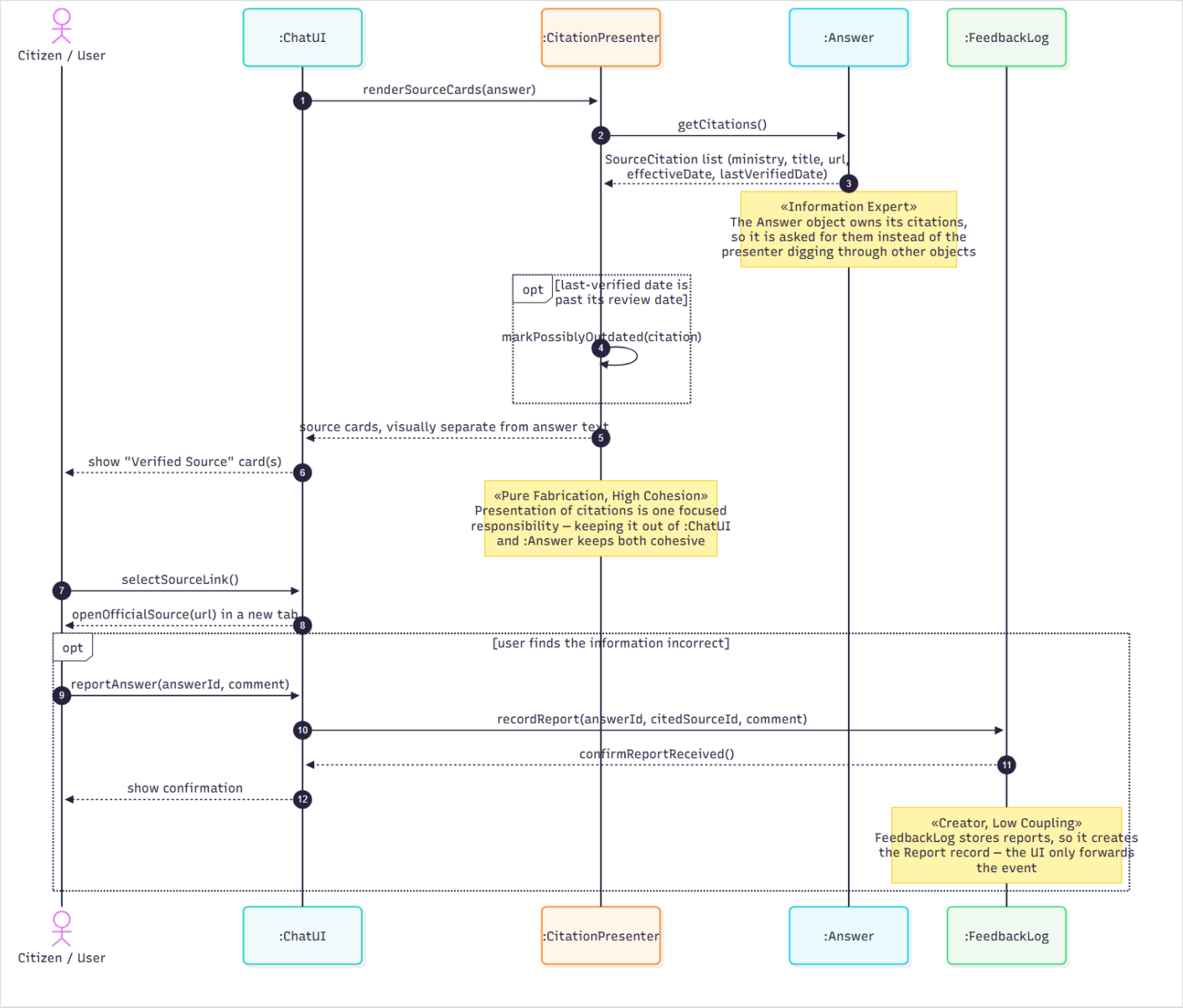


Figure 3: Sequence diagram for UC-3: ViewSourceCitation.

Design principles: the Answer object supplies its own SourceCitation list, so no other object needs to dig through its structure (**Information Expert**). CitationPresenter is a **Pure Fabrication** whose single responsibility is rendering source cards visually separate from the

answer text (**High Cohesion**, REQ-28). FeedbackLog creates and stores the Report record when the user flags an answer, so the UI only forwards the event (**Creator**, **Low Coupling**).

UC-7: GetOfficeContactAndLocation

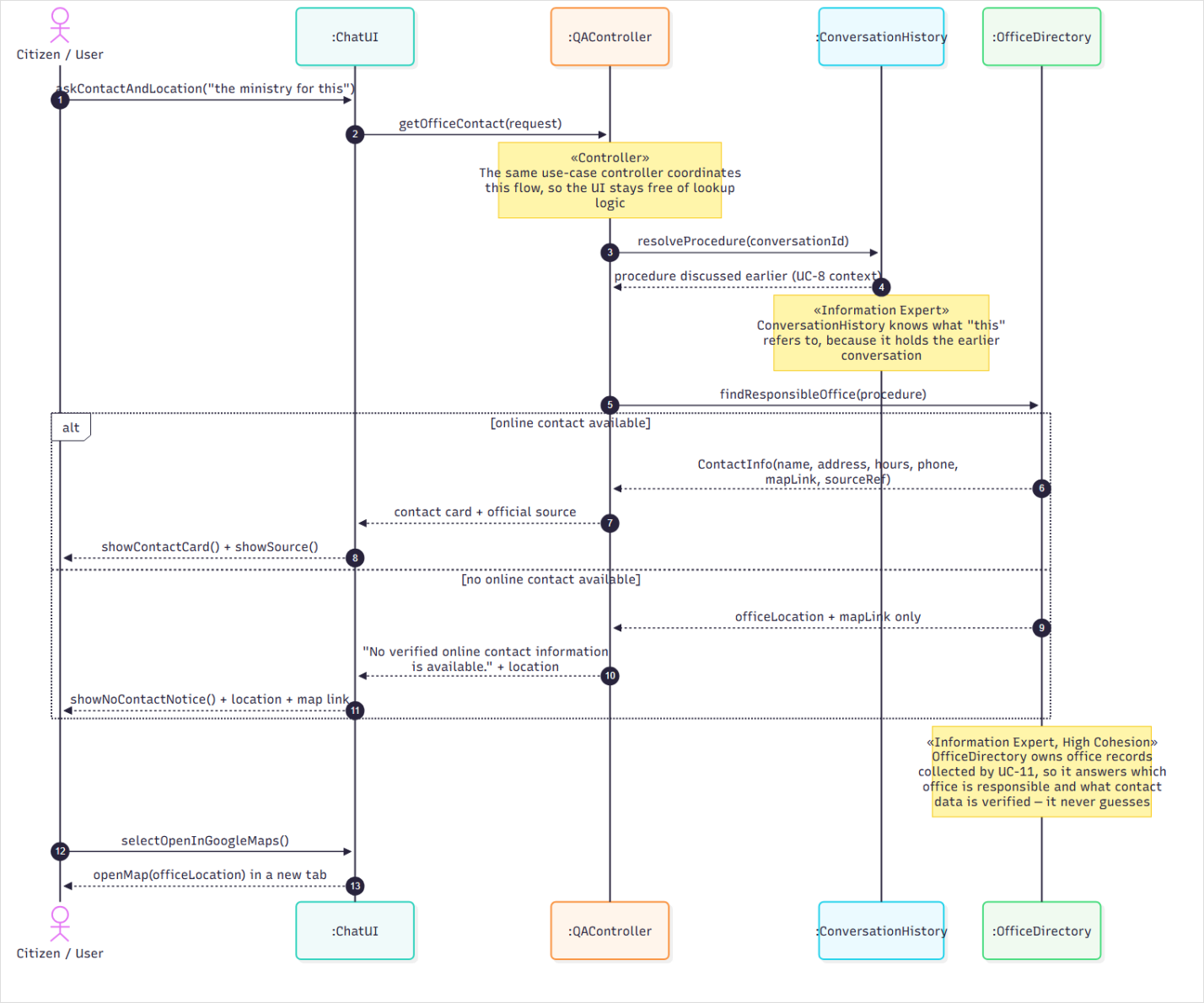


Figure 4: Sequence diagram for UC-7: GetOfficeContactAndLocation.

Design principles: QAController again acts as the use-case **Controller**, keeping lookup logic out of the UI. ConversationHistory resolves what “this” refers to because it holds the earlier conversation (**Information Expert**). OfficeDirectory owns the verified office records collected by UC-11, so it alone answers which office is responsible and what contact data is verified —

when nothing is verified it says so rather than guessing (**Information Expert**, **High Cohesion**, REQ-13/REQ-39).

UC-11: RefreshOfficialContent

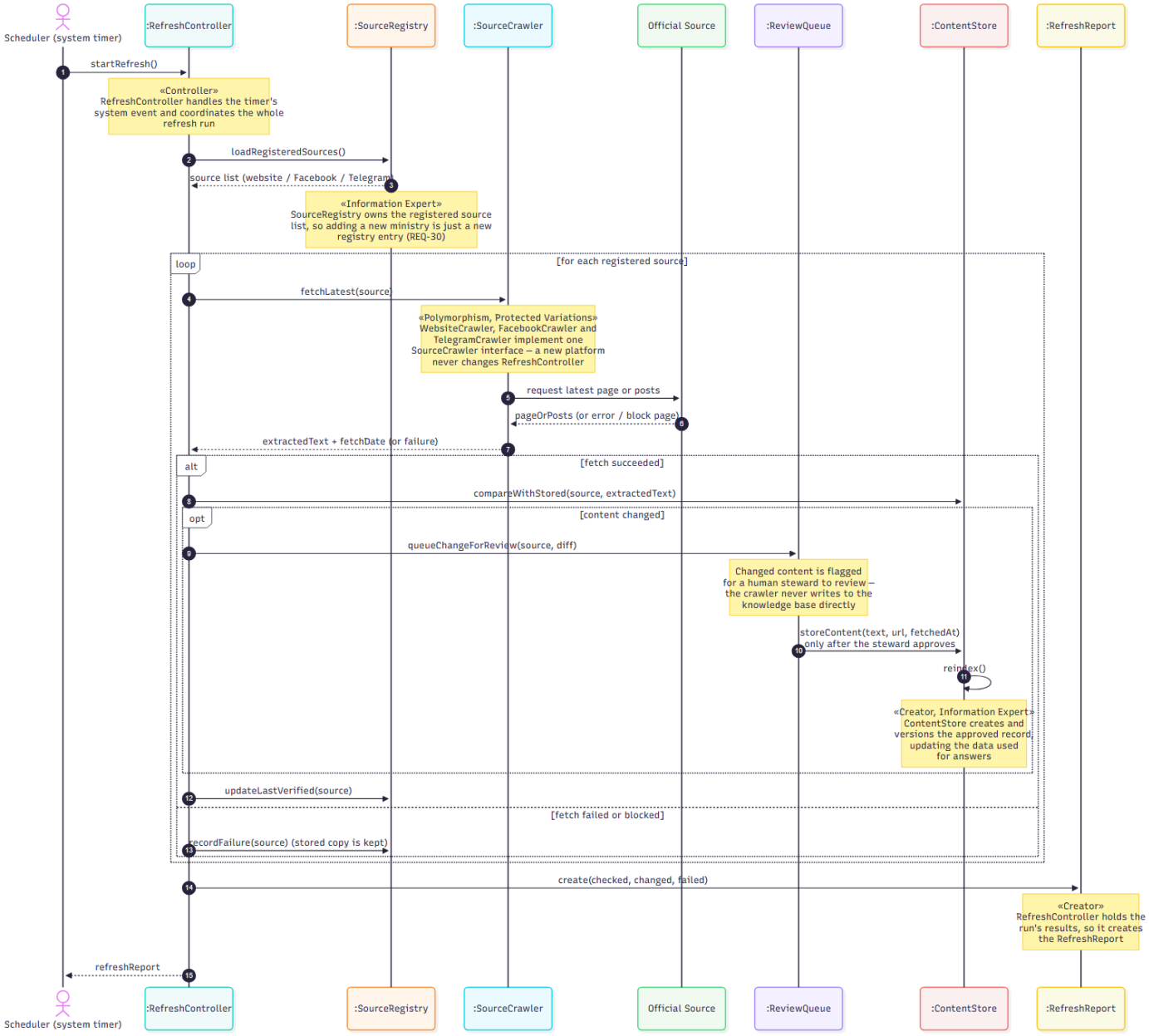


Figure 5: Sequence diagram for UC-11: RefreshOfficialContent.

Design principles: RefreshController handles the Scheduler's system event and coordinates the run (**Controller**). SourceRegistry owns the registered source list, so adding a new ministry is just a new registry entry with no change to the rest of the system (**Information Expert**, REQ-30). The platform crawlers (website, Facebook, Telegram) implement one SourceCrawler interface,

so a new platform never changes RefreshController (**Polymorphism**, **Protected Variations**). Changed content is queued for a human steward to review — the crawler never writes to the knowledge base directly. ContentStore creates and versions the approved content records because it aggregates them (**Creator**, **Information Expert**), and RefreshController creates the RefreshReport from the results it holds (**Creator**).

2. Class Diagram and Interface Specification

a. Class Diagram

The class diagram refines the design objects that appeared as lifelines in the sequence diagrams of Section 1 (Figures 1–5) into a single static model. Every lifeline there is represented as a class here, and the messages exchanged in those diagrams are reflected in the class operations. Where a lifeline represents a family of implementations (the two search indexes behind SearchIndex, the model vendors behind LlmProvider, and the platform crawlers behind SourceCrawler), the family is shown explicitly as an interface with its implementations. The classes are also checked against the implemented prototype in the project repository (lib/engine, lib/retrieval, lib/llm, lib/kb, lib/crawl, lib/log, components/chat), so the names in this section correspond to those in the code.

Because the full model has 36 classes, it is presented according to the course requirements. Figure 6 provides an overview showing every class and its associations, using only the class-name compartment and grouping the classes into six packages. Figures 7–10 are partial diagrams showing the three compartments (name, attributes, and operations) for the classes associated with each use-case group.

In the diagrams only the **visibility** of each member is shown (**+** public, **−** private); the data types and full operation signatures are given in Section 2b. Associations are read in the direction of the arrow; a hollow triangle with a dashed line is interface realisation; a filled diamond is composition (the part cannot exist without the whole); a hollow diamond is aggregation; a dashed arrow labelled *creates* is a dependency where the source constructs the target.

How the partial diagrams fit into the overview. The six packages of Figure 6 are the key. *Boundary* (ChatUI, TelegramBot) and *Control* (QAController, RefreshController) sit on the left of the overview; the citizen-facing partial diagrams A, B and C all start from them. *Services* and *SearchAndModel* are split between partial A (LanguageService, RetrievalService, SearchIndex and its two indexes) and partial B (AIModelGateway, LlmProvider and its providers, AnswerVerifier, CitationPresenter). *KnowledgeAndConversation* is spread over A (ContentStore, Passage, Candidate, ConversationHistory, Conversation, Question), B (Answer, SourceCitation, FeedbackLog, Report) and C (OfficeDirectory, ContactInfo). *ContentRefresh* is partial D in full. Classes that appear in more than one partial diagram are the joins between them: **QAController**

appears in A, B and C; **ChatUI** in A, B and C; **ContentStore** in A and D (retrieval reads it, the refresh writes it after steward approval); **ConversationHistory** in A and C; **Passage** in A and B; and **RegisteredSource** in C and D. Where a class is repeated, the later diagram shows only the members that matter for that use case, and the full member list is the one in the first diagram that shows it.

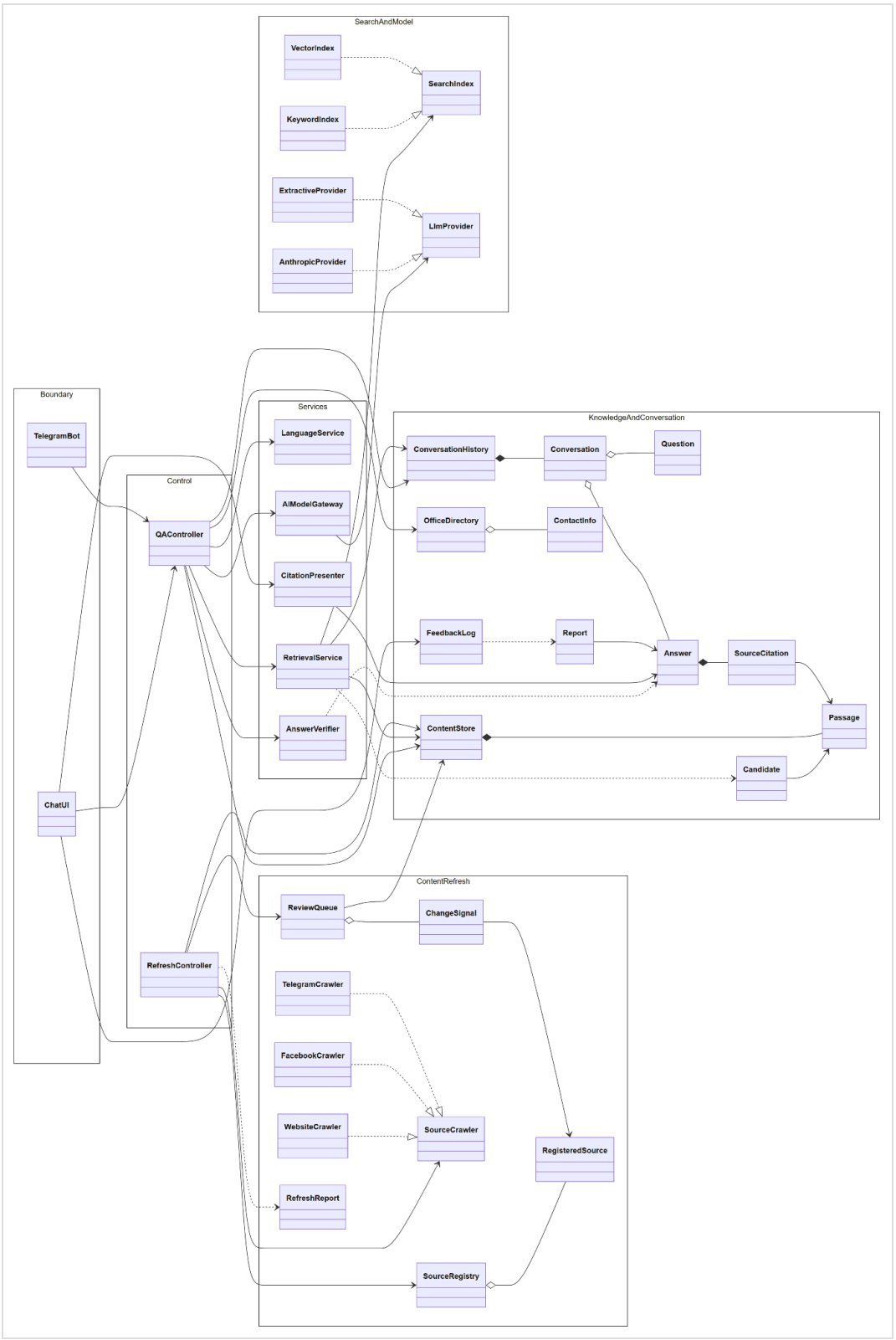


Figure 6: Overview class diagram of SuperAsk.AI. All 36 classes and their associations, name compartment only, grouped into six packages.

The overview follows the four responsibility-assignment decisions of Part One. The two **Controller** classes are the only entry points from the boundary: everything the web chat or the Telegram bot does passes through QAController, and everything the Scheduler starts passes through RefreshController. The **Information Expert** classes (ContentStore, ConversationHistory, SourceRegistry, OfficeDirectory) own the data they answer questions about. The **Pure Fabrication** services (LanguageService, RetrievalService, AnswerVerifier, CitationPresenter) hold behaviour that belongs to no entity. The three interfaces (SearchIndex, LlmProvider, SourceCrawler) are the **Protected Variations** points: a new search method, model vendor, or source platform is a new realisation of one interface and nothing else changes, which is what REQ-29 and REQ-30 require.

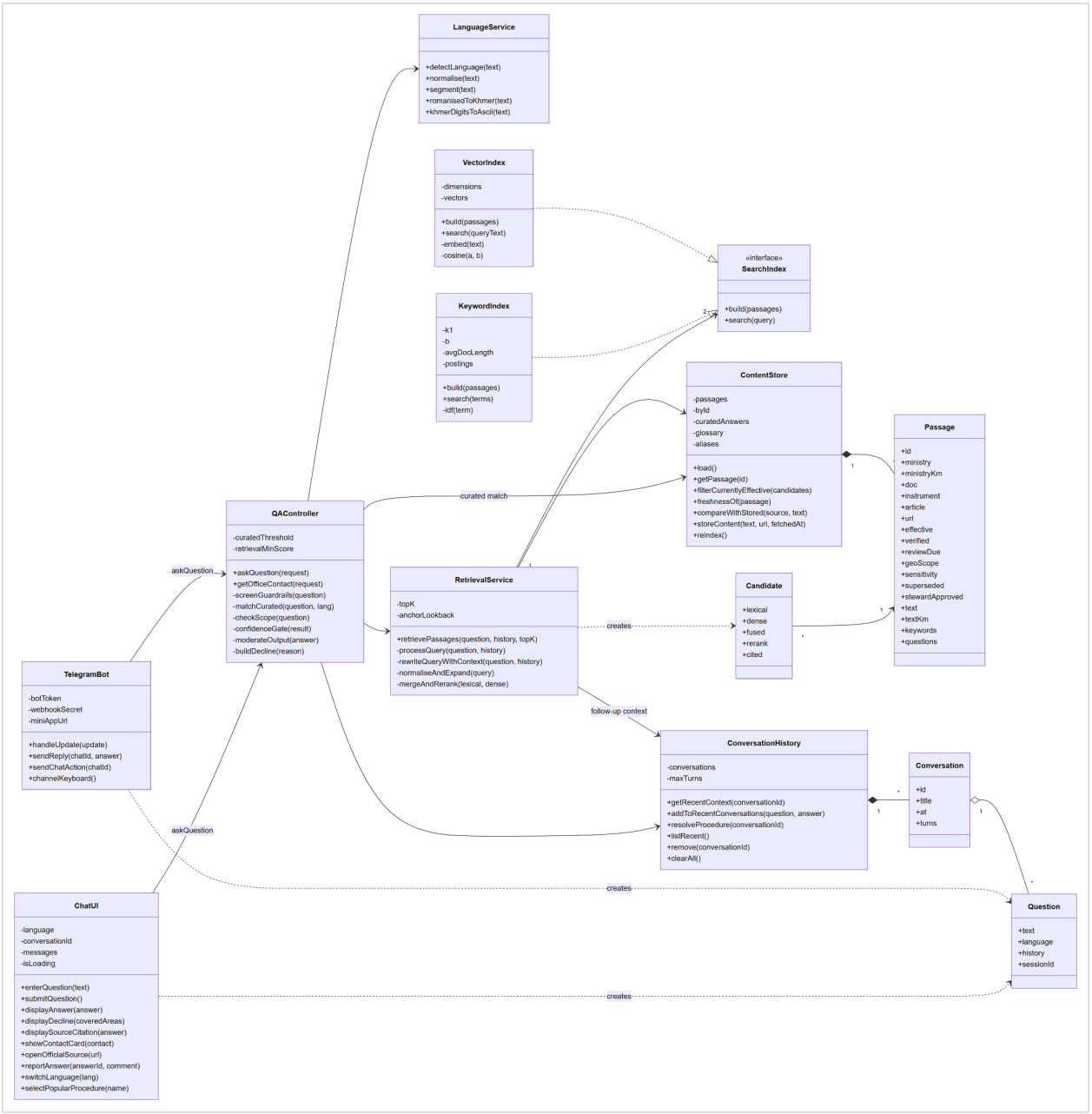


Figure 7: Partial class diagram A: question handling and retrieval (UC-1 AskQuestion, UC-2 RetrieveOfficialSources, UC-6, UC-8, UC-9, UC-10).

ChatUI and TelegramBot are the two boundary classes (REQ-1, REQ-15); both create a Question and hand it to QAController. QAController keeps the two thresholds that decide the flow: a curated answer is served word-for-word when the match score is at least *curatedThreshold*, and retrieval results below *retrievalMinScore* trigger UC-4 instead of generation (REQ-7). LanguageService handles Khmer/English detection and normalisation (REQ-2, REQ-3). RetrievalService drives exactly two SearchIndex realisations, a KeywordIndex (BM25) and a VectorIndex (embeddings), fuses and reranks their results into Candidates, and asks ContentStore

to drop passages that are no longer in effect (REQ-4, REQ-5). ConversationHistory stores Conversations for UC-6 and supplies the context RetrievalService needs to rewrite a follow-up question (UC-8, REQ-14).

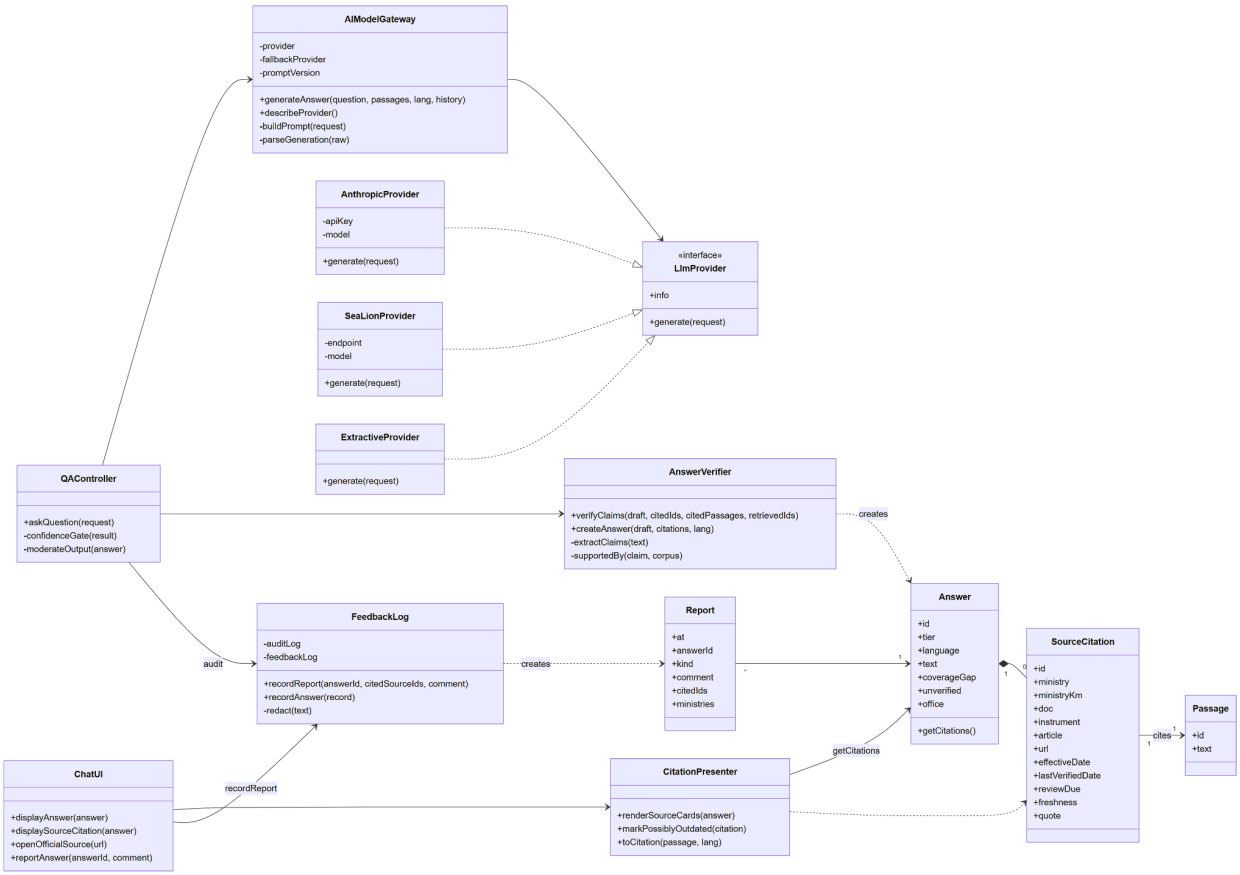


Figure 8: Partial class diagram B: answer generation, verification, citation and feedback (UC-1, UC-3 ViewSourceCitation, UC-4).

AIModelGateway is the only class that talks to an AI model; it selects one LlmProvider realisation (AnthropicProvider or SeaLionProvider) and falls back to ExtractiveProvider, which quotes the best passage verbatim, when the model service is unavailable (UC-1 extension 7a). AnswerVerifier checks every fee, date and document in the draft against the cited passages before it creates the Answer (REQ-22); an Answer is composed of its SourceCitations, each of which cites exactly one Passage (REQ-6). CitationPresenter renders the citations as cards visually separate from the answer text and flags any citation past its review date (REQ-28, UC-3 extension 2a). FeedbackLog creates a Report when the citizen flags an answer and also writes the audit record for every answer served.

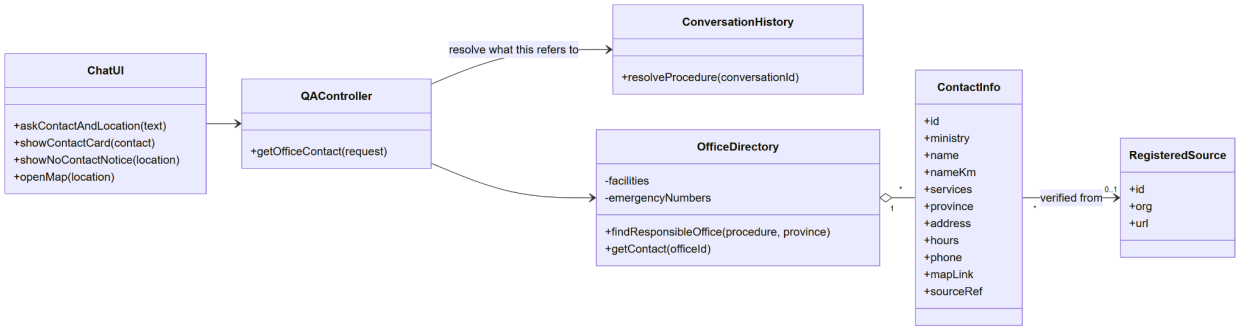


Figure 9: Partial class diagram C: office contact and location (UC-7 GetOfficeContactAndLocation).

QAController asks ConversationHistory which procedure “this” refers to, then asks OfficeDirectory for the responsible office. OfficeDirectory aggregates the ContactInfo records (today loaded from *data/facilities.json*) and also holds the emergency numbers. Each ContactInfo may point to the RegisteredSource it was verified from (REQ-11, REQ-12); when no verified online contact exists, OfficeDirectory returns the location only and the ChatUI shows the “no verified online contact” notice (REQ-13, REQ-39).



Figure 10: Partial class diagram D: scheduled content refresh with steward review (UC-11 RefreshOfficialContent).

RefreshController loads the RegisteredSources from SourceRegistry, calls *fetchLatest* on the SourceCrawler realisation matching each source's platform, and compares what comes back with ContentStore. A detected change becomes a ChangeSignal in the ReviewQueue; only when a human steward approves it does ReviewQueue call *storeContent* on ContentStore and reindex. The crawler classes never write to ContentStore directly, which is the design rule that keeps machine-fetched text out of citizen answers. RefreshController creates the RefreshReport from

the counts it accumulated during the run (sources attempted and reachable, pages fetched, blocked by robots.txt, rejected as block pages, changes, errors).

b. Data Types and Operation Signatures

The specification below is written independently of the diagrams, in UML notation, with one table per class. Attribute types use the UML primitives (*String*, *Integer*, *Real*, *Boolean*, *Date*) plus *List<T>* and *Map<K,V>*; enumerations are written as a set of literals. A few small value types are shared by several classes and are defined once first.

Shared value types

Type	Definition	Meaning
Lang	enumeration {km, en}	Language of a question, answer, or interface text.
Tier	enumeration {1, 2, 3}	How the answer was produced: 1 curated human-approved answer, 2 retrieval-augmented generation, 3 refusal.
Freshness	enumeration {fresh, due, stale}	Whether a passage's last-verified date is within, at, or past its review date.
Confidence	enumeration {high, medium, low}	The model's self-reported confidence; low confidence forces a decline.
Turn	structure {role: enumeration {user, assistant}, text: String}	One message of a conversation, used as history context.
RetrievalResult	structure {query: ProcessedQuery, candidates: List<Candidate>, topScore: Real}	What RetrievalService returns; topScore is the best rerank score, normalised 0–1, and drives the confidence gate.
ProcessedQuery	structure {raw: String, normalised: String, segments: List<String>, expanded: List<String>, romanised: Boolean, rewritten: String}	The question after normalisation, segmentation, alias expansion and (for follow-ups) rewriting.
GenerationRequest	structure {question: String, lang: Lang, mode: enumeration {grounded, general}, sources: List<Passage>, history: List<Turn>}	Everything a model provider needs to write an answer; *grounded* is the default and restricts the model to the supplied passages.

GenerationResult	structure {answer: String, citations: List<String>, confidence: Confidence, shouldDecline: Boolean}	The draft answer with the passage ids it rests on.
VerificationResult	structure {passed: Boolean, unsupported: List<String>, invalidCitations: List<String>}	Outcome of the claim check: claims with no supporting passage, and cited ids that were never retrieved.
GuardrailHit	structure {kind: String, message: String}	Why a question or answer was stopped by a guardrail (emergency, legal advice, political, out of scope, moderation).
FetchResult	structure {status: Integer, body: String, fetchedAt: Date, blocked: Boolean, error: String}	What a crawler got back from an official source.
CrawlOptions	structure {mode: enumeration {verify, monitor}, priorities: List<Integer>}	Which sources a refresh run covers; the weekly run uses mode monitor and priorities 1 and 2.

ChatUI — *boundary*. The web chat interface (React components under *components/chat*). It collects the question, shows the answer, citations, contact card or decline notice, and forwards reports. It holds no business logic.

Attribute	Type	Meaning
language	Lang	Interface language chosen by the user (UC-10).
conversationId	String	Id of the conversation currently shown.
messages	List<Turn>	Turns displayed in the chat pane.
isLoading	Boolean	True while a question is being answered; shows the loading indicator.

Operation	Meaning
+enterQuestion(text: String): void	Puts the typed text into the input field; empty text keeps the send button disabled (UC-1 2a).
+submitQuestion(): void	Builds a Question from the input and the recent turns and calls QAController.askQuestion.

+displayAnswer(answer: Answer): void	Renders the answer text in the question's language with steps, documents, fees, deadlines and hours.
+displayDecline(coveredAreas: List<String>): void	Shows the “cannot answer” message with the areas SuperAsk.AI does cover (UC-4).
+displaySourceCitation(answer: Answer): void	Asks CitationPresenter to render the Verified Source cards under the answer (UC-3).
+showContactCard(contact: ContactInfo): void	Shows name, address, hours, phone and the Google Maps link (UC-7).
+openOfficialSource(url: String): void	Opens the cited official page in a new tab.
+reportAnswer(answerId: String, comment: String): void	Forwards a citizen's “this is incorrect” report to FeedbackLog.
+switchLanguage(lang: Lang): void	Switches interface text between Khmer and English (UC-10).
+selectPopularProcedure(name: String): void	Sends a sidebar procedure as a question (UC-5).

TelegramBot — *boundary*. The Telegram channel (webhook route plus *lib/telegram*). It receives updates from the Telegram Bot API, turns them into Questions for QAController, and sends the answer back as a message. It also offers the Mini App button that opens ChatUI inside Telegram (UC-9).

Attribute	**Type**	**Meaning**
botToken	String	Bot API credential, held server-side only (REQ-27).
webhookSecret	String	Secret Telegram must present on every webhook call.
miniAppUrl	String	Public URL of ChatUI embedded as the Mini App.

Operation	**Meaning**
+handleUpdate(update: TelegramUpdate): void	Validates the secret, extracts chat id and text, appends the turn to ConversationHistory and calls QAController.askQuestion.

+sendReply(chatId: Integer, answer: Answer): void	Formats the Answer and its citations as a Telegram message and sends it.
+sendChatAction(chatId: Integer): void	Shows the “typing” indicator while the answer is produced.
+channelKeyboard(): ReplyMarkup	Builds the keyboard with the Open SuperAsk (Mini App) button.

Question — *entity*. A citizen's question as submitted through any channel (the *AskRequest* contract in the code).

Attribute	Type	Meaning
text	String	The question in Khmer, English or mixed.
language	Lang	Detected by LanguageService; answers come back in this language.
history	List<Turn>	Recent turns of the same conversation, for follow-up resolution (UC-8).
sessionId	String	Correlates the question with its audit record.

No operations (data class; accessed through its attributes).

QAController — *control*. Use-case controller for all citizen-facing flows (UC-1, 2, 3, 4, 7, 8). Runs the fixed pipeline guardrail screen → curated match → retrieval → confidence gate → generation → verification → output moderation → citation assembly (*lib/engine/tiers.ts*).

Attribute	Type	Meaning
curatedThreshold	Real	Minimum match score for a Tier 1 curated answer to be served verbatim (0.72).
retrievalMinScore	Real	Decline floor: if the top rerank score is below it the question goes to UC-4 (0.20).

Operation	Meaning
+askQuestion(request: Question): Answer	Main operation of UC-1; returns a sourced Answer, a curated answer, or a decline.

+getOfficeContact(request: Question): Answer	UC-7: resolves the procedure, finds the office and returns an Answer whose *office* field holds the ContactInfo.
−screenGuardrails(question: String): GuardrailHit	Policy screen that runs before any retrieval (emergency, legal advice, political, chit-chat); null when the question may proceed.
−matchCurated(question: String, lang: Lang): CuratedMatch	Looks for a human-approved Tier 1 answer; null when none scores above curatedThreshold.
−checkScope(question: String): Boolean	True when the question is about a government service (UC-1 5a).
−confidenceGate(result: RetrievalResult): Boolean	True when result.topScore ≥ retrievalMinScore; false sends the flow to UC-4.
−moderateOutput(answer: String): GuardrailHit	Checks the generated text before it is shown; null when clean.
−buildDecline(reason: String): Answer	Builds the Tier 3 Answer with the decline text, covered areas and coverageGap = true.

RefreshController — *control*. Use-case controller for UC-11. Started by the Scheduler (a weekly scheduled task), it coordinates the crawl of every registered source, change detection, queueing for steward review, and the run report (*lib/crawl/crawl.ts*).

Attribute	**Type**	**Meaning**
mode	enumeration {verify, monitor}	verify probes that a source is reachable; monitor fetches content and detects changes.
priorities	List<Integer>	Which source priorities this run covers (weekly run: 1 and 2).
startedAt	Date	Start time, copied into the RefreshReport.

Operation	**Meaning**
+startRefresh(options: CrawlOptions): RefreshReport	Runs the whole refresh and returns the report.

−compareWithStored(source: RegisteredSource, snapshot: SourceSnapshot): List<ChangeSignal>	Compares fetched text hashes with the stored snapshot; one ChangeSignal per new, modified, removed or unreachable page.
−classifyChange(diff: String): enumeration {factual, presentation}	Keyword scan for fees, deadlines, documents, eligibility and steps; factual changes always need steward approval.
−createReport(): RefreshReport	Creates the RefreshReport from the counts gathered during the run.

LanguageService — *service*. Khmer/English handling in one place (*lib/lang*, *lib/khmer*): detection, Unicode normalisation, segmentation and romanised-Khmer mapping.

No attributes.

Operation	Meaning
+detectLanguage(text: String): Lang	km when the text contains Khmer script, otherwise en.
+normalise(text: String): String	Unicode normalisation and canonical reordering of Khmer clusters; case folding for Latin.
+segment(text: String): List<String>	Splits Khmer (no spaces) and English text into search terms.
+romanisedToKhmer(text: String): String	Maps romanised Khmer typed on a Latin keyboard to Khmer script.
+khmerDigitsToAscii(text: String): String	Converts Khmer numerals so fees and dates compare correctly.

RetrievalService — *service*. Hybrid retrieval over the approved content (*lib/retrieval*): query processing, keyword and vector search, reciprocal-rank fusion, reranking and the effective-content filter (UC-2).

Attribute	Type	Meaning
topK	Integer	Number of reranked candidates returned (8).
anchorLookback	Integer	How many earlier turns are searched for the subject of a follow-up question (4).

Operation	Meaning
+retrievePassages(question: String, history: List<Turn>, topK: Integer): RetrievalResult	Main operation of UC-2; returns the ranked candidates with their passages and the top score.
−processQuery(question: String, history: List<Turn>): ProcessedQuery	Normalises, segments, expands aliases and, for an anaphoric follow-up, rewrites the question.
−rewriteQueryWithContext(question: String, history: List<Turn>): String	Replaces “it”/“this” with the subject named in an earlier turn (UC-2 2a).
−normaliseAndExpand(query: String): List<String>	Adds known synonyms and abbreviations such as MPWT.
−mergeAndRerank(lexical: Map<String,Real>, dense: Map<String,Real>): List<Candidate>	Reciprocal-rank fusion of the two score maps followed by a cross-encoder style rerank.

SearchIndex — *interface*. Common interface of the two search methods, so RetrievalService does not depend on either (Polymorphism, Protected Variations). Both realisations take the same ProcessedQuery and each reads the part of it that it needs, so RetrievalService calls them identically.

No attributes.

Operation	Meaning
+build(passages: List<Passage>): void	Indexes the given passages.
+search(query: ProcessedQuery): Map<String,Real>	Returns passage id → score for the processed query.

KeywordIndex — *class (realises SearchIndex)*. BM25 keyword index (*lib/retrieval/bm25.ts*).

Attribute	Type	Meaning
k1	Real	BM25 term-frequency saturation parameter.
b	Real	BM25 length-normalisation parameter.
avgDocLength	Real	Average passage length in terms.

postings	Map<String, Map<String,Integer>>	term → (passage id → term frequency).

Operation	**Meaning**
+build(passages: List<Passage>): void	Builds the postings from passage text, keywords and candidate questions.
+search(query: ProcessedQuery): Map<String,Real>	BM25 score per passage, using query.segments and query.expanded as the search terms.
−idf(term: String): Real	Inverse document frequency of a term.

VectorIndex — *class (realises SearchIndex)*. Meaning-based index over passage embeddings (*lib/retrieval/embed.ts*).

Attribute	**Type**	**Meaning**
dimensions	Integer	Embedding size.
vectors	Map<String, List<Real>>	passage id → embedding.

Operation	**Meaning**
+build(passages: List<Passage>): void	Embeds every passage.
+search(query: ProcessedQuery): Map<String,Real>	Embeds query.rewritten (or query.normalised when there is no rewrite) and returns the cosine similarity to each passage.
−embed(text: String): List<Real>	Produces the embedding of a text.
−cosine(a: List<Real>, b: List<Real>): Real	Cosine similarity of two vectors.

AIModelGateway — *service*. The only class that calls an external AI model (*lib/llm*). Chooses the configured LlmProvider, builds the versioned grounding prompt, parses the JSON reply and falls back to ExtractiveProvider on failure (Indirection, Protected Variations).

Attribute	Type	Meaning
provider	LlmProvider	Configured model provider.
fallbackProvider	LlmProvider	ExtractiveProvider, used when the configured provider fails or times out (UC-1 7a).
promptVersion	String	Version of the grounding prompt, logged with every answer (grounding-v6).

Operation	Meaning
+generateAnswer(question: String, passages: List<Passage>, lang: Lang, history: List<Turn>): GenerationResult	Writes an answer using only the supplied passages and returns the ids it cited.
+describeProvider(): ProviderInfo	Provider id, model, hosting and data residency, for diagnostics.
−buildPrompt(request: GenerationRequest): String	Assembles the system prompt, the SOURCES block and the JSON contract.
−parseGeneration(raw: String): GenerationResult	Parses the model's JSON reply; a malformed reply counts as a failure.

LlmProvider — *interface*. Common interface of all model providers.

Attribute	Type	Meaning
info	ProviderInfo	Static description of the provider (id, label, model, hosting, residency).

Operation	Meaning
+generate(request: GenerationRequest): GenerationResult	Produces a draft answer for the request.

AnthropicProvider — *class (realises LlmProvider)*. Provider that calls the Anthropic Claude API.

Attribute	Type	Meaning

apiKey	String	API credential, server-side only (REQ-27).
model	String	Model identifier.

Operation	Meaning
+generate(request: GenerationRequest): GenerationResult	Sends the prompt to the API and parses the reply.

SeaLionProvider — *class (realises LlmProvider)*. Provider for a self-hosted or regional model endpoint (SEA-LION / vLLM), kept for data-residency experiments.

Attribute	Type	Meaning
endpoint	String	Base URL of the model server.
model	String	Model identifier.

Operation	Meaning
+generate(request: GenerationRequest): GenerationResult	Sends the prompt to the endpoint and parses the reply.

ExtractiveProvider — *class (realises LlmProvider)*. Deterministic fallback that needs no credential: it returns the most relevant retrieved passage word for word, with its citation.

No attributes.

Operation	Meaning
+generate(request: GenerationRequest): GenerationResult	Returns the top passage's text as the answer and cites it.

AnswerVerifier — *service*. Checks that every fee, date, document and number in the draft appears in a cited passage, then creates the Answer (*lib/engine/verify.ts*; REQ-22).

No attributes.

Operation	Meaning
+verifyClaims(draft: String, citedIds: List<String>, citedPassages: List<Passage>, retrievedIds: List<String>): VerificationResult	Returns passed = false with the unsupported claims when a claim has no source, or when a cited id was never retrieved.
+createAnswer(draft: String, citations: List<SourceCitation>, lang: Lang): Answer	Creates the final Answer object (Creator).
−extractClaims(text: String): List<String>	Pulls amounts, dates, durations and document names out of the draft.
−supportedBy(claim: String, corpus: String): Boolean	True when the claim (or its digit/word equivalent) occurs in the cited text.

Answer — *entity*. A response shown to the citizen (the *AskResponse* contract).

Attribute	Type	Meaning
id	String	Server-assigned id used by Report and audit records.
tier	Tier	How the answer was produced.
language	Lang	Language of the answer text.
text	String	The answer, or the decline text for Tier 3.
coverageGape	Boolean	True when the question is in scope but no approved source covers it (UC-4).
unverified	Boolean	True only when the operator-enabled general-knowledge fallback produced the answer; citations are then empty and the interface shows an unverified marker.
office	ContactInfo	Responsible office, present for UC-7 answers.

Operation	Meaning

+getCitations(): List<SourceCitation>	The citations that support this answer, in the order cited (Information Expert).

SourceCitation — *entity*. One Verified Source card under an answer (the *Citation* contract; REQ-6, REQ-28).

Attribute	Type	Meaning
id	String	Id of the cited Passage.
ministry	String	Ministry in English.
ministryKm	String	Ministry in Khmer.
doc	String	Document or page title.
instrument	String	Legal instrument number.
article	String	Article or annex reference.
url	String	Link to the official page.
effectiveDate	Date	When the content took effect.
lastVerifiedDate	Date	When the source was last checked.
reviewDue	Date	Date after which the citation is flagged as possibly outdated.
freshness	Freshness	Derived from lastVerifiedDate and reviewDue.
quote	String	The passage excerpt the answer rests on.

No operations (data class; accessed through its attributes).

CitationPresenter — *service*. Renders citations as source cards separate from the answer text (*SourceCard*, *FreshnessBadge*, *toCitation*).

No attributes.

Operation	Meaning

+renderSourceCards(answer: Answer): void	One card per citation, in citation order (UC-3 2b).
+markPossiblyOutdated(citation: SourceCitation): void	Adds the “confirm with the office” note when freshness is stale (UC-3 2a).
+toCitation(passage: Passage, lang: Lang): SourceCitation	Builds a SourceCitation from a Passage, choosing the Khmer or English fields.

FeedbackLog — *service*. Append-only logs of answers served and citizen reports (*lib/log/audit.ts*). Personal data is redacted before writing.

Attribute	Type	Meaning
auditLog	String	Path of the answer audit log (var/audit.jsonl).
feedbackLog	String	Path of the report log (var/feedback.jsonl).

Operation	Meaning
+recordReport(answerId: String, citedSourceIds: List<String>, comment: String): Report	Creates and stores a Report against the cited sources (Creator; UC-3 3a).
+recordAnswer(record: AuditRecord): void	Logs question, language, tier, provider, prompt version, retrieved and cited ids and latency for every answer.
−redact(text: String): String	Masks phone numbers, ids and other personal data.

Report — *entity*. A citizen's report about an answer (the *FeedbackRecord* contract).

Attribute	Type	Meaning
at	Date	When the report was made.
answerId	String	The reported Answer.
kind	enumeration {incorrect, helpful, unhelpful}	What the citizen said about the answer.
comment	String	Optional free text, redacted.

citedIds	List<String>	Passages the answer cited, for steward routing.
ministries	List<String>	Ministries that own the cited content.

No operations (data class; accessed through its attributes).

ContentStore — *entity / expert*. The approved knowledge base (*lib/kb/loader.ts* over *data/kb*, *curated.json*, *glossary.json*, *aliases.json*). It owns the passages and their effective, verified and review dates, so it decides what is currently in effect.

Attribute	Type	Meaning
passages	List<Passage>	All approved passages.
byId	Map<String,Passage>	Lookup by passage id.
curatedAnswers	List<CuratedAnswer>	Tier 1 human-approved answers.
glossary	List<GlossaryTerm>	Official terminology in Khmer and English.
aliases	Map<String, List<String>>	Colloquial term → official terms, for query expansion.

Operation	Meaning
+load(): void	Reads the data files and builds both search indexes.
+getPassage(id: String): Passage	Returns one passage.
+filterCurrentlyEffective(candidates: List<Candidate>): List<Candidate>	Drops superseded passages and passages whose effective date is in the future (UC-2 step 4).
+freshnessOf(passage: Passage): Freshness	fresh, due or stale from the verified and review dates.
+compareWithStored(source: RegisteredSource, text: String): Boolean	True when the fetched text differs from the stored copy (UC-11 step 6).
+storeContent(text: String, url: String, fetchedAt: Date): Passage	Creates a new passage version with its source link and fetch date, marking the previous one superseded (Creator).
+reindex(): void	Rebuilds the indexes after content changes.

Passage — *entity*. One unit of approved official content: an article, provision or page section with full provenance (the *Chunk* contract).

Attribute	Type	Meaning
id	String	Unique id.
ministry	String	Owning ministry, English.
ministryKm	String	Owning ministry, Khmer.
doc	String	Source document title.
instrument	String	Legal instrument number.
article	String	Article or annex.
url	String	Link to the original.
effective	Date	Date the content took effect.
verified	Date	Date last verified.
reviewDue	Date	Date after which it is flagged for review.
geoScope	String	Province code, or KH for nationwide.
sensitivity	enumeration {public, internal, restricted}	Only public passages are served to citizens.
superseded	Boolean	True for replaced versions; excluded from retrieval.
stewardApproved	Boolean	True when a content steward approved it; false for anything machine-ingested.
text	String	English text.
textKm	String	Khmer text.
keywords	List<String>	Maintained alias set for retrieval.
questions	List<String>	Candidate citizen questions indexed alongside the passage.

No operations (data class; accessed through its attributes).

Candidate — *entity*. A retrieval result: one Passage with its component and fused scores.

Attribute	Type	Meaning
lexical	Real	Keyword (BM25) score.
dense	Real	Vector similarity score.
fused	Real	Reciprocal-rank-fusion score.
rerank	Real	Final rerank score, 0–1.
cited	Boolean	True when the answer cited this passage.

No operations (data class; accessed through its attributes).

ConversationHistory — *entity / expert*. Recent conversations of one citizen. On the web it lives in the browser (*lib/ui/history.ts*, localStorage); for Telegram it is a short per-chat memory on the server (*lib/telegram/history.ts*).

Attribute	Type	Meaning
conversations	Map<String, Conversation>	Conversations by id.
maxTurns	Integer	Turns kept per Telegram chat (8).

Operation	Meaning
+getRecentContext(conversationId: String): List<Turn>	Earlier turns, for follow-up rewriting (UC-8).
+addToRecentConversations(question: Question, answer: Answer): void	Appends the exchange and updates the conversation title and time (UC-1 step 11).
+resolveProcedure(conversationId: String): String	The procedure most recently discussed, for “the ministry for this” (UC-7 step 2).
+listRecent(): List<Conversation>	Most recent first, for the sidebar (UC-6).

+remove(conversationId: String): void	Deletes one conversation.
+clearAll(): void	Deletes all conversations.

Conversation — *entity*. One conversation thread.

Attribute	Type	Meaning
id	String	Unique id.
title	String	Derived from the first question.
at	Date	Time of the last turn.
turns	List<Turn>	Questions and answers in order.

No operations (data class; accessed through its attributes).

OfficeDirectory — *entity / expert*. Verified office records (*data/facilities.json* loaded by *lib/kb/loader.ts*), plus emergency numbers.

Attribute	Type	Meaning
facilities	List<ContactInfo>	Offices with their services and provinces.
emergencyNumbers	List<EmergencyNumber>	Label (km/en) and number.

Operation	Meaning
+findResponsibleOffice(procedure: String, province: String): ContactInfo	The office that handles the procedure in that province; null when nothing verified exists (UC-7 4a, 4b).
+getContact(officeId: String): ContactInfo	One office record.

ContactInfo — *entity*. Contact and location of one office (the *Facility* contract).

Attribute	Type	Meaning
id	String	Unique id.

ministry	String	Parent ministry.
name	String	Office name, English.
nameKm	String	Office name, Khmer.
services	List<String>	Procedures handled here.
province	String	Province.
address	String	Street address.
hours	String	Office hours.
phone	String	Phone number.
mapLink	String	Google Maps link.
sourceRef	String	Id of the RegisteredSource the details were verified from.

No operations (data class; accessed through its attributes).

SourceRegistry — *entity / expert*. The registered official sources (*data/sources.json*, generated from the registry CSV). Adding a ministry is a new entry here and nothing else (REQ-30).

Attribute	**Type**	**Meaning**
sources	List<RegisteredSource>	All registered sources, 132 today.

Operation	**Meaning**
+loadRegisteredSources(): List<RegisteredSource>	Enabled sources, for a refresh run (UC-11 step 2).
+register(source: RegisteredSource): void	Adds a source.
+updateLastVerified(source: RegisteredSource): void	Sets lastVerified to now after a successful fetch (UC-11 step 8).

+recordFailure(source: RegisteredSource, error: String): void	Records an unreachable or blocked source; the stored copy and previous date are kept (UC-11 4a).

RegisteredSource — *entity*. One official website, Facebook page or Telegram channel SuperAsk.AI monitors (the *Source* contract).

Attribute	Type	Meaning
id	String	Unique id.
org	String	Organisation, English.
orgKm	String	Organisation, Khmer.
category	String	Ministry, agency, provincial office, etc.
url	String	Seed URL.
host	String	Host name, for politeness delays.
platform	enumeration {web, facebook, telegram, video, document}	Which SourceCrawler handles it.
priority	Integer	1 citizen-facing (crawled weekly) to 3 peripheral.
depth	Integer	Link-following depth from the seed.
maxPages	Integer	Page cap per run.
politenessDelayMs	Integer	Minimum delay between requests to this host (2000 ms default).
enabled	Boolean	False excludes the source from runs.
lastVerified	Date	Last successful fetch.
kbMinistry	String	Ministry name in the knowledge base this source feeds.

No operations (data class; accessed through its attributes).

SourceCrawler — *interface*. Common interface of the platform crawlers, so RefreshController never changes when a platform is added (Polymorphism, Protected Variations).

No attributes.

Operation	Meaning
+fetchLatest(source: RegisteredSource): FetchResult	Fetches the latest page or posts of the source.

WebsiteCrawler — *class (realises SourceCrawler)*. Crawler for ministry websites (*lib/crawl/fetch.ts*, *robots.ts*, *extract.ts*, *soft-block.ts*). Implemented.

Attribute	Type	Meaning
userAgent	String	SuperAskBot/0.1 with a contact mailbox, sent on every request.
timeoutMs	Integer	Per-request timeout (20 000 ms).
maxBytes	Integer	Body size cap (5 MB).

Operation	Meaning
+fetchLatest(source: RegisteredSource): FetchResult	Checks robots.txt, waits for the host's politeness delay, fetches, detects block pages and extracts text.
−checkRobots(host: String): Boolean	False when robots.txt disallows automated access (UC-11 3a).
−extractText(html: String): String	Removes menus and repeated page elements (UC-11 step 5).
−detectSoftBlock(response: FetchResult): Boolean	True for a login or WAF page returned with status 200, treated as failure not change (UC-11 4b).

FacebookCrawler — *class (realises SourceCrawler)*. Crawler for public Facebook pages. Planned adapter; registered sources with platform facebook are skipped until it exists.

No attributes.

Operation	Meaning

+fetchLatest(source: RegisteredSource): FetchResult	Fetches the latest public posts.

TelegramCrawler — *class (realises SourceCrawler)*. Crawler for public Telegram channels. Planned adapter.

No attributes.

Operation	Meaning
+fetchLatest(source: RegisteredSource): FetchResult	Fetches the latest channel posts.

ReviewQueue — *entity / expert*. Changes waiting for a human steward (*var/review-queue.jsonl*, *lib/crawl/store.ts*). The only path from fetched content into ContentStore.

Attribute	Type	Meaning
queuePath	String	Path of the queue file.

Operation	Meaning
+queueChangeForReview(changes: List<ChangeSignal>): void	Appends detected changes as pending.
+pendingCount(): Integer	Number of changes awaiting review.
+approve(change: ChangeSignal): void	Steward action: calls ContentStore.storeContent and reindex, then marks the change approved. Portal for this is planned.

ChangeSignal — *entity*. One detected change at an official source.

Attribute	Type	Meaning
id	String	Unique id.
sourceId	String	The RegisteredSource.
url	String	Page that changed.

kind	enumeration {new, modified, removed, unreachable}	What happened.
risk	enumeration {factual, presentation}	Factual changes (fees, deadlines, documents, steps) always need approval.
riskTerms	List<String>	The terms that triggered the factual class.
detectedAt	Date	When detected.
title	String	Page title.
previousRawPath	String	Retained copy before the change.
currentRawPath	String	Retained copy after the change.
status	enumeration {pending, approved, rejected}	Review state; always pending when created.

No operations (data class; accessed through its attributes).

RefreshReport — *entity*. Summary of one refresh run (the *CrawlReport* contract; UC-11 step 9).

Attribute	**Type**	**Meaning**
startedAt	Date	Run start.
finishedAt	Date	Run end.
sourcesAttempted	Integer	Sources in the run.
sourcesReachable	Integer	Sources that answered.
pagesFetched	Integer	Pages received.
pagesBlockedByRobots	Integer	Pages skipped because of robots.txt.
pagesRejected	Integer	Responses discarded as block pages.

changes	structure {total, factual, presentation: Integer, byKind: Map<String,Integer>}	Change counts.
errors	List<structure {sourceId: String, error: String}>	Failures by source.

No operations (data class; accessed through its attributes).

c. Traceability Matrix

Report One did not contain a separate domain model, so the domain concepts traced here are the nouns of Report One itself: the things named in the use-case descriptions (Section 3c), the fully-dressed use cases (3c-iv), the system sequence diagrams (3d) and the requirements (Section 2). Each row names one such concept, the design class or classes it became, and why the name or the number of classes changed. Classes that appear in no row (KeywordIndex, VectorIndex, Candidate, AnthropicProvider, SeaLionProvider, ExtractiveProvider, WebsiteCrawler, FacebookCrawler, TelegramCrawler) are realisations or working data that exist only because of a design decision, and they are explained with the concept they serve.

Traceability (1): domain concepts of Report One to design classes

Domain concept (Report One)	**Design class(es)**	**Why the name or the number of classes changed**
Citizen's question (UC-1 steps 1–2; REQ-1, REQ-2, REQ-3, REQ-31, REQ-32)	Question, ChatUI, TelegramBot	The question itself became the Question entity. The “chat interface” and “messaging channel” of REQ-1 and REQ-15 became two boundary classes, ChatUI and TelegramBot, because they share nothing but the Question they create and the Answer they display. Messenger (REQ-15) would be a third boundary class with the same two responsibilities.
SuperAsk.AI as a black box (3d system sequence diagrams)	QAController, RefreshController, LanguageService, RetrievalService, AIModelGateway, AnswerVerifier, CitationPresenter	The black box was opened in Part One into one controller per initiating actor (QAController for the citizen, RefreshController for the Scheduler) and the Pure Fabrication services that hold the behaviour the black box performed in steps 4–9 of UC-1: language detection (LanguageService), lookup (RetrievalService), writing (AIModelGateway), checking (AnswerVerifier) and displaying sources (CitationPresenter). Splitting them keeps each class cohesive and makes the pipeline order in QAController visible.
Language detection and Khmer handling (UC-1 step 4;	LanguageService; Lang type	Named after its single responsibility. The *language* value became a shared Lang type carried by Question, Answer and ChatUI rather than an attribute of one class.

REQ-2, REQ-3, REQ-26, REQ-37)		
Stored official content / official passages (UC-2 steps 2–6; REQ-4, REQ-5, REQ-29)	ContentStore, Passage	One concept became two classes: Passage is the unit of content with its provenance (ministry, instrument, article, effective and verified dates), and ContentStore is the Information Expert that holds all passages and decides which are currently in effect. REQ-29 is why the content is data in the store, not code.
Keyword search and meaning-based search (UC-2 step 2)	SearchIndex (interface), KeywordIndex, VectorIndex, Candidate	The two search methods of the use case became two realisations of one SearchIndex interface so RetrievalService depends on neither. Candidate did not exist as a domain concept; it is the working record that carries the two scores, the fused score and the rerank score through step 3 (merge and re-rank) to the confidence gate.
AI Model Service (participating actor of UC-1; UC-1 step 7, 7a)	AIModelGateway, LlmProvider (interface), AnthropicProvider, SeaLionProvider, ExtractiveProvider	The external actor stays external; what the system owns is the gateway that talks to it. The gateway was given a provider interface with three realisations so the model vendor can change without touching the pipeline (Protected Variations), and so the 7a fallback (“show the most relevant passage word for word”) is just another provider, ExtractiveProvider.
Answer with steps, documents, fees, deadlines, hours (UC-1 step 9; REQ-16 to REQ-20, REQ-33)	Answer	Kept as one entity. Tier, coverageGap and unverified were added because the use-case extensions (curated answer, UC-4 decline, model fallback) are all Answers of a different kind rather than different classes.
Verified source / citation (UC-3; REQ-6, REQ-28, REQ-34)	SourceCitation, CitationPresenter	The “Verified Source card” concept split into the data (SourceCitation, composed into Answer) and the behaviour of showing it separately from the answer text and flagging an outdated one (CitationPresenter). The split keeps REQ-28 in one class.
Claim check (UC-1 step 8, 8a; REQ-22)	AnswerVerifier	Became its own class because it is the safety gate of the whole system and is tested on its own; it also creates the Answer because it already holds the draft and the citations (Creator).
Recent conversations and conversation	ConversationHistory, Conversation, Turn type	The list of recent conversations became ConversationHistory (the expert that lists, stores and resolves references) holding Conversation entities made of Turns. One class serves both

context (UC-6, UC-8; REQ-10, REQ-14, REQ-36)		the web (browser storage) and Telegram (server memory), which is why it is an abstraction rather than a UI component.
Ministry / responsible office, contact and location (UC-7; REQ-11, REQ-12, REQ-13, REQ-20, REQ-38, REQ-39)	OfficeDirectory, ContactInfo	The office became ContactInfo (the record) and OfficeDirectory (the expert that finds the responsible office and knows when nothing verified exists, so it never guesses). The Google Maps link became the mapLink attribute.
Report of incorrect information (UC-3 3a)	Report, FeedbackLog	The citizen's report became a Report entity created by FeedbackLog. FeedbackLog also absorbed the audit trail of answers, which Report One mentions only indirectly through REQ-22 acceptance testing, because they are the same append-only, redacted log mechanism.
Registered official sources: ministry websites, Facebook pages, Telegram channels (UC-11 precondition, steps 2–4; REQ-8, REQ-30)	SourceRegistry, RegisteredSource, SourceCrawler (interface), WebsiteCrawler, FacebookCrawler, TelegramCrawler	The source list became SourceRegistry (expert) holding RegisteredSource entries. The three platforms named in Report One became three realisations of one SourceCrawler interface, so a fourth platform is one new class. Only WebsiteCrawler is implemented so far; the other two are in the diagram because the use case names them.
Comparing with the stored copy and saving changed text (UC-11 steps 6–7)	ChangeSignal, ReviewQueue, ContentStore.storeContent	Report One let the system save changed text directly. The design inserts a human step: a change becomes a ChangeSignal in a ReviewQueue, and only an approved change reaches ContentStore. This was added because the implementation review found that a crawler fetching a page is not a steward approving it, and it is the mechanism behind the “steward review” note in the UC-11 sequence diagram.
Refresh report (UC-11 step 9)	RefreshReport	Kept as one entity, created by RefreshController, with the counts the use case lists plus the robots and block-page counts from extensions 3a and 4b.
Scheduler (initiating actor of UC-11)	no class	Remains an actor outside the system: a weekly scheduled task on a team machine that runs RefreshController.startRefresh. It owns no data and no behaviour of SuperAsk.AI.

The second matrix reads the other way, from design classes to the use cases of Report One (Section 3c-i), so that every class can be seen to serve at least one use case and the five demonstration use cases (UC-1, 2, 3, 7, 11) can be seen to be covered end to end. QAController, ContentStore, ConversationHistory and Answer appear in the most columns because they are the shared backbone of the citizen-facing flows; RefreshController and the refresh entities appear only in UC-11, which is why the refresh runs as a separate process from the web server (Section 3).

Traceability (2): design classes to use cases

Design class	**UC -1**	**UC -2**	**UC -3**	**UC -4**	**UC -5**	**UC -6**	**UC -7**	**UC -8**	**UC -9**	**UC-10**	**UC-11**
ChatUI	X		X	X	X	X	X	X		X	
TelegramBot									X		
Question	X				X			X	X		
QAController	X	X		X			X	X			
LanguageService	X									X	
RetrievalService		X						X			
SearchIndex / KeywordIndex / VectorIndex		X									
ContentStore	X	X									X
Passage		X	X								X
Candidate		X									
ConversationHistory / Conversation	X					X	X	X			
AIModelGateway / LlmProvider and providers	X										
AnswerVerifier	X										
Answer	X		X	X			X		X		

SourceCitation	X		X								
CitationPresenter			X								
FeedbackLog / Report	X		X	X							
OfficeDirectory / ContactInfo				X			X				
RefreshController											X
SourceRegistry / RegisteredSource							X				X
SourceCrawler and crawlers											X
ReviewQueue / ChangeSignal											X
RefreshReport											X

3. System Architecture

a. Identifying Subsystems

1. UML Package Diagram

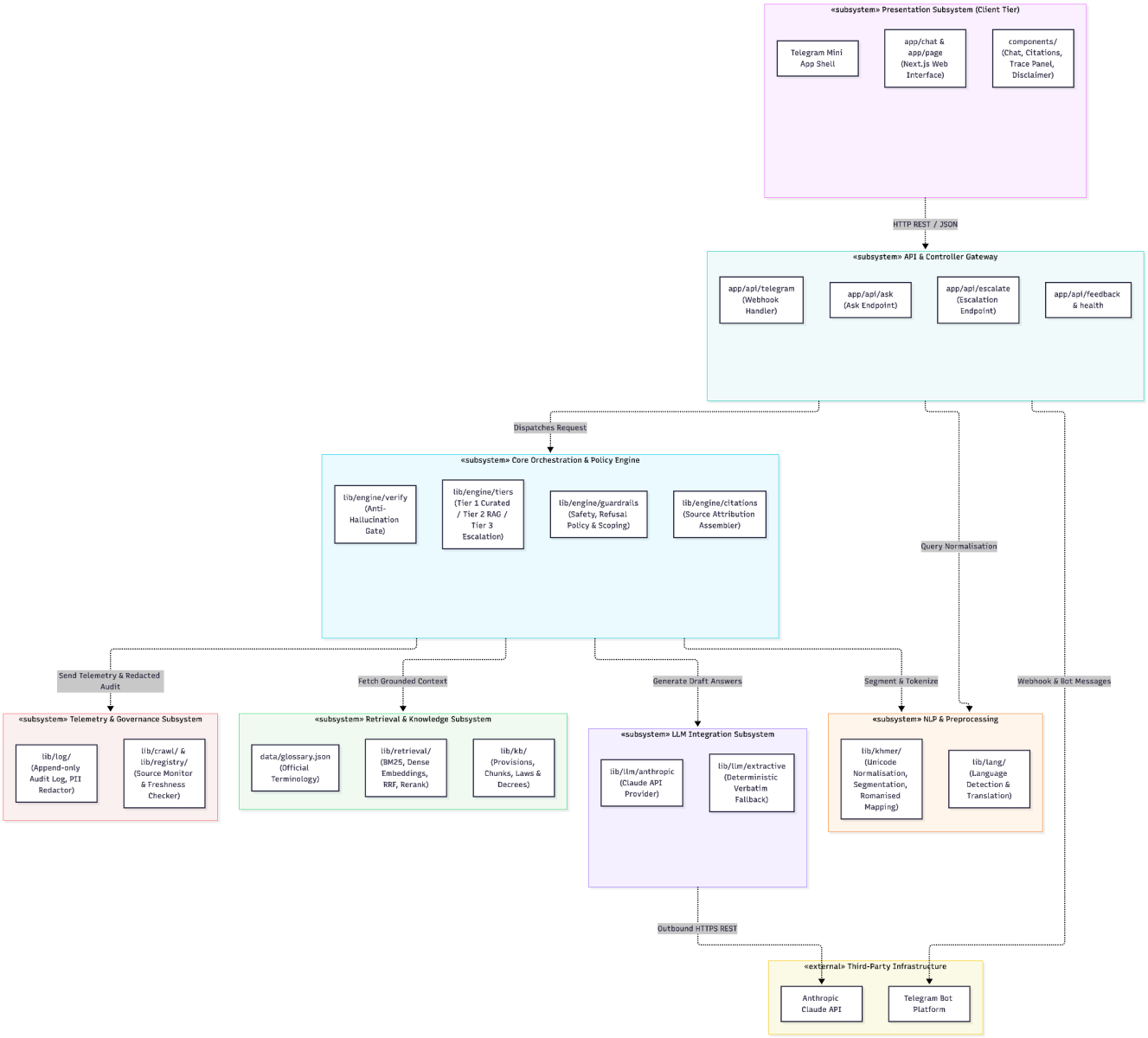


Figure 11: UML Package Diagram.

2. Description of Subsystems

1. Presentation Subsystem (Client Tier):

- Responsibility: Provides the responsive, bilingual (English & Khmer) user interface for citizens.

- Key Modules: Web Chat page (/chat), landing page, UI components (components/) including message history, source citation cards, verification badges, and developer trace panels. It also powers the Telegram Mini App view.

2. API Gateway & Controller Subsystem:

- Responsibility: Serves as the public entry point for incoming network traffic.
- Key Modules: Next.js App Router endpoints (/api/ask, /api/feedback, /api/health, and /api/telegram). It handles JSON payload validation, CORS, rate limits, and dispatching to backend pipelines.

3. Natural Language Processing (NLP) Subsystem:

- Responsibility: Resolves linguistic challenges specific to Khmer and mixed-language citizen queries.
- Key Modules: lib/khmer/ (Unicode normalisation, zero-width space canonical reordering, dictionary-based segmentation, and Romanised-Khmer transliteration mapping) and lib/lang/ (language identification between km and en).

4. Retrieval & Knowledge Subsystem:

- Responsibility: Manages domain knowledge and retrieves authoritative legal provisions without hallucinations.
- Key Modules: lib/retrieval/ implementing hybrid search (lexical BM25 + dense vector embeddings) fused through Reciprocal Rank Fusion (RRF) and cross-encoder reranking; lib/kb/ containing legal chunks, sub-decrees, and ministry documents; and data/glossary.json for official government terminology.

5. Core Orchestration & Policy Guardrail Subsystem:

- Responsibility: The central brain executing the safety and answer pipeline.
- Key Modules:
    - Tier Routing (lib/engine/tiers.ts): Routes to Tier 1 (Curated FAQ / Glossary), Tier 2 (RAG grounded answer), or Tier 3 (Withhold & Decline).
    - Guardrails (lib/engine/guardrails.ts): Enforces strict refusal policies on emergency calls, legal dispute advice, political queries, and out-of-scope tasks.

    - Verification Gate (lib/engine/verify.ts): Cross-checks generated figures, fees, and dates against retrieved source texts before showing them to the citizen.

6. Model Integration Subsystem (LLM Abstraction):

- Responsibility: Encapsulates language model providers behind a uniform interface.
- Key Modules: lib/llm/anthropic.ts (invoking Claude models) and lib/llm/extractive.ts (a deterministic, zero-cost extractive provider that quotes retrieved text verbatim when API credentials are absent or fail).

7. Audit & Governance Subsystem:

- Responsibility: Maintains system integrity, security, and continuous legal compliance.
- Key Modules: Append-only query audit logger with automatic PII redaction (lib/log/) and content monitoring (lib/crawl/, lib/registry/) that checks source ministries for stale or updated regulations.

b. Architecture Styles

The system combines four primary architectural styles to satisfy high availability, bilingual processing, safety guardrails, and cloud deployment:

1. **Client-Server Architecture**
    - Description: Separates user interface presentation from core processing logic and data storage.
    - Application in System:
    - Client: Web browsers and the Telegram Mini App render the visual chat interface, accept user input, manage local session states, and display citations.
    - Server: The Next.js Node.js / Serverless runtime processes requests, runs NLP, queries knowledge databases, and interfaces with AI models.
    - Advantage: Allows multi-platform access (web, mobile, Telegram) while keeping sensitive knowledge data and LLM secrets securely on the server.
2. **Layered (N-Tier) Architecture**
    - Description: Structures software into distinct horizontal layers where each layer has a dedicated responsibility and only communicates with adjacent layers.

- Application in System:
- Presentation Layer: React UI components, chat interfaces, and theme providers.
- API / Controller Layer: Next.js App Router API endpoints (/api/ask, /api/telegram).
- Domain / Orchestration Layer: Guardrails, tier router, verification gate, and citation builder.
- Service & Retrieval Layer: Khmer text segmentation, BM25 / RRF hybrid search, and LLM abstraction adapters.
- Data / Knowledge Layer: Legal chunk registry, glossary entries, and append-only audit files.
- Advantage: High modularity, ease of testing (600+ automated unit/integration tests), and separation of concerns.

3. **Pipeline / Pipes-and-Filters Architecture (RAG Pipeline)**
    - Description: Data flows sequentially through a chain of processing components (filters), where the output of one component serves as the input to the next.
    - Application in System: The Ask Engine follows a strict, verifiable linear processing pipeline: Query Input → Unicode Normalisation → Policy Guardrails → Curated Answer Match → Hybrid Retrieval → Confidence Gate → Prompt Assembly → LLM Generation → Verification Gate → Citation Linking → Final Response
    - Advantage: If any filter fails (e.g., policy filter flags an emergency query, or the verification gate detects an unverified fee), execution halts immediately and routes to a safe fallback or decline without executing subsequent costly steps.
4. **Adapter / Facade Pattern (Service-Oriented Integration)**
    - Description: Provides unified wrapper interfaces around heterogeneous external systems.
    - Application in System:
    - lib/llm/ provides a pluggable provider interface hiding whether answers are generated via Anthropic Claude API or via the local Deterministic Extractive Fallback.
    - lib/telegram/ provides an adapter bridging the Telegram Bot Webhook API into internal standard AskRequest contracts.

c. Mapping Subsystems to Hardware

The system is a distributed cloud application that executes across multiple independent physical and virtual machines. The subsystem names used in the table and in the deployment diagram are the seven subsystems defined in Section 3a; each subsystem is

assigned to exactly one machine, and the connectors between the machines are described in Section 3d.

Hardware Mapping Table

Hardware Machine / Environment	Description & Infrastructure	Subsystems Running on this Machine
Machine 1: Client Tier (End-User Devices)	Citizen smartphones, tablets, laptops, and desktop computers.	• Presentation Subsystem (Web Browser, React 19 / Next.js Client Bundle) • Telegram Client / Mini App container UI
Machine 2: Application Server (current deployment: Vercel serverless Node.js; proposed production: GCP Cloud Run / Compute Engine, see Section 3f)	Distributed serverless edge and Node.js compute instances hosted in the cloud.	• API Gateway & Controller Subsystem (/api/*) • Natural Language Processing (NLP) Subsystem • Core Orchestration & Policy Guardrail Subsystem • Retrieval & Knowledge Subsystem (in-memory indexes over the static knowledge base) • Model Integration Subsystem (LLM adapters, which call Machine 3) • Audit & Governance Subsystem: audit logging with PII redaction (its content-monitoring part runs on Machine 5)
Machine 3: External AI Platform (Anthropic GPU Cloud Cluster)	High-performance AI hardware acceleration clusters (GPU/TPU clusters managed by Anthropic).	• Foundation LLM inference (Claude models), called over HTTPS by the Model Integration Subsystem on Machine 2
Machine 4: Messaging Cloud Server (Telegram Cloud Infrastructure)	Worldwide distributed messaging servers managed by Telegram.	• Telegram Bot API server and webhook relay, connecting the Telegram client on Machine 1 to the API Gateway & Controller Subsystem on Machine 2

Machine 5: Content-Monitoring Workstation (team computer)	Windows PC with Node.js 24 that runs the weekly crawl as a scheduled task (Sunday 02:00 ICT) in a separate process from the web server; it reaches the official sources over HTTP(S).	• Audit & Governance Subsystem, content-monitoring part (lib/crawl, lib/registry): started by the Scheduler, it raises change signals to the review queue and never writes to the knowledge base

Hardware Deployment Diagram

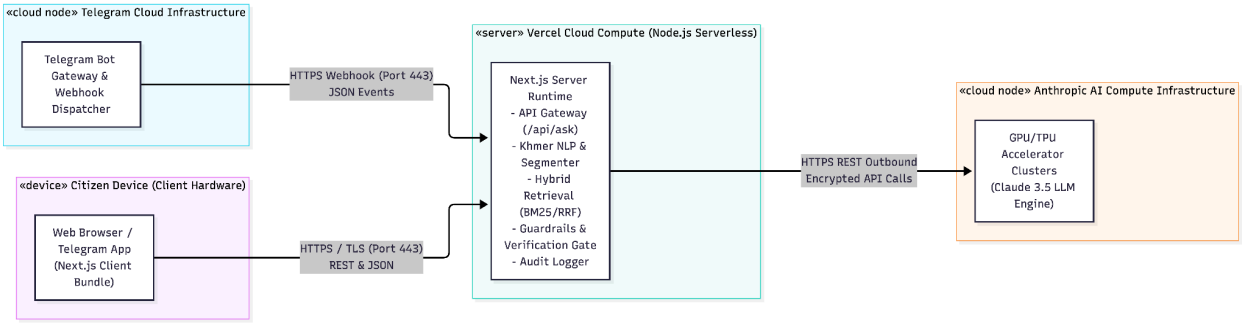


Figure 12: Hardware deployment diagram: the five machines of the hardware mapping table and the connectors between them.

d. Connectors and Network Protocols

SuperAsk.AI requires network communication between the user interfaces, backend services, and external platforms. The system currently supports the web application, Telegram bot, and Telegram Mini App, while Messenger integration is planned as an additional communication channel. These interfaces are designed to use the same core question-and-answer system.

The web interface communicates with the backend through a question-and-answer API over HTTPS with JSON bodies (POST /api/ask, POST /api/feedback and GET /api/health); request and response bodies are limited to 4.5 MB by the host. When a user submits a question, QAController coordinates the request, RetrievalService retrieves relevant government information, and AIModelGateway communicates with the external AI model service. The system then returns an answer with supporting source citations. Telegram uses the same backend functionality: the Telegram Bot API delivers each message by HTTPS webhook to POST /api/telegram/webhook, which checks a shared

secret, and replies go back through the Bot API over HTTPS; the Mini App is the same /chat page loaded inside Telegram, which the page allows as a frame ancestor. The AI model is called over HTTPS JSON through the Anthropic SDK. Messenger is intended to follow the same webhook approach once integrated.

The system also connects to official government information sources. For the content-refresh process (UC-11), RefreshController coordinates information retrieval from registered government websites, Facebook pages, and Telegram channels through the SourceCrawler interface. The website crawler, the review queue and the weekly scheduled run are implemented today, while the Facebook and Telegram crawlers and the steward-approval portal are planned. The crawler uses HTTP(S) GET with a SuperAskBot user agent, respects robots.txt and waits 2 seconds between requests to the same host. New or updated information is queued as a change signal and must be approved by a steward before being added to the knowledge base.

These connections allow SuperAsk.AI to support multiple platforms while keeping backend processing separate from the user interfaces. API keys and other sensitive credentials remain on the server side, as required by REQ-27.

e. Global Control Flow

- *Execution orderness*

SuperAsk.AI mainly follows an event-driven control flow rather than a fixed linear sequence. The system responds to actions initiated by users, such as submitting questions, viewing source citations, and requesting government office information.

For example, when a user submits a question through the web interface or Telegram, QAController receives the request and coordinates the necessary operations. The system screens the question against the policy guardrails, checks for a curated Tier 1 answer, retrieves relevant information using RetrievalService, applies the confidence gate, generates a response through AIModelGateway, and verifies the answer with AnswerVerifier before presenting it with source citations, the same pipeline order as the class design in Section 2 and the pipes-and-filters style in Section 3b. ConversationHistory supports follow-up questions by maintaining the context of previous interactions.

Users can perform different actions depending on their needs without following the same sequence each time. In addition to user-triggered operations, the system design includes a scheduled content-refresh process controlled by RefreshController. This process is initiated automatically by the Scheduler rather than by a user: it runs as a weekly Windows scheduled task (Sunday 02:00 ICT, scripts/schedule-weekly.ps1) on a team

computer, in a separate process from the web server, so the two flows never compete for the same resources.

- *Time dependency*

  SuperAsk.AI is primarily an interactive request-response system and does not have strict real-time execution requirements. However, response time is important because users expect to receive information without unnecessary delays. According to REQ-24, under normal testing conditions, the system should begin displaying a response within 5 seconds for at least 90% of test queries.

  The system also includes a periodic content-refresh process (UC-11: RefreshOfficialContent). The Scheduler initiates this process at configured intervals to check registered official sources for updated government information. New or changed content is then reviewed before being added to the approved knowledge base.

  Therefore, SuperAsk.AI combines event-driven interactions with a time-based background process. The host caps a web request at 300 seconds and the Telegram webhook at 60 seconds; the 5-second target of REQ-24 is dominated by the model call, since retrieval runs in milliseconds. Although the system has a response-time performance target, it is not classified as a hard real-time system with strict deadlines for every operation.

f. Hardware Requirements

SuperAsk.AI is designed to operate as a web-based application that can be accessed through computers and mobile devices. The system requires client devices, server resources, data storage, and internet connectivity to support communication between users and backend services.

On the client side, users need a computer, laptop, tablet, or smartphone with at least 2 GB of RAM and a 3G, 4G, or Wi-Fi connection. Users can access SuperAsk.AI through a compatible web browser or Telegram, while Messenger is planned as an additional communication channel. According to REQ-25, the web interface should remain usable without horizontal scrolling on screens approximately 360 pixels wide through standard desktop resolutions. The interface must also display Khmer and English text correctly, as specified in REQ-26.

On the server side there are two configurations. The current prototype is deployed on Vercel's serverless platform (Machine 2 in Section 3c): each request runs in a Node.js 24 function with 1 vCPU, 2 GB of RAM and a 300-second limit (60 seconds for the Telegram webhook), the deployment bundle must stay under 250 MB, and the knowledge base (664 KB on disk: 45 approved passages from four organisations and 132 registered sources) is loaded from disk and indexed in memory when a function starts, using well under 100 MB. For the production deployment the team proposes Google Cloud Platform (GCP), with Google Cloud Run or Compute Engine (e2-standard-2) as hosting options. The proposed server configuration includes 2–4 virtual CPUs (vCPUs), 4–8 GB of RAM, and 50 GB of SSD storage using Balanced Persistent Disk. The proposed network configuration includes a 1 Gbps network interface with a minimum dedicated throughput target of 50 Mbps. The preferred hosting region is asia-southeast1 (Singapore) to support connectivity for users in Cambodia. The subsystem mapping in Section 3c does not change between the two configurations: the same Machine 2 subsystems run on either platform.

These resources are intended to support the web application, question-and-answer API, information retrieval, and communication with external AI services. Storage is also needed for government-service information, verified source records, and application data; the audit and feedback logs are append-only files that are not persistent on the serverless host, so the production deployment must keep them in a database or log service. The weekly content-refresh run (Machine 5 in Section 3c) needs a separate team computer with Node.js 24, 2 GB of RAM and about 1 GB of disk that grows with the retained copies of fetched pages (at most 5 MB per page); with a 20-second timeout per request and a 2-second politeness delay per host, the 132 registered sources complete well within the task's 4-hour limit. API keys and other sensitive credentials must remain on the server side, as specified in REQ-27.

4. Algorithms and Data Structures

a. Algorithms

Report One did not define a formal mathematical model. Instead, it described the retrieval steps in UC-2: RetrieveOfficialSources (keyword search, meaning-based search, merging and re-ranking the results, and removing outdated content) and the answer checks in UC-1: AskQuestion. The algorithms below implement those steps.

SuperAsk.AI uses several algorithms to process user questions, retrieve relevant government information, and generate responses supported by official sources. These algorithms follow the system architecture described in Section 3 and are represented by the classes and operations in

Section 2. The main algorithms support UC-1: AskQuestion, UC-2: RetrieveOfficialSources, and UC-11: RefreshOfficialContent.

1. Natural Language Processing

When a citizen submits a question, LanguageService first identifies whether the text is in Khmer or English. The system then normalizes the input, divides it into searchable terms, and expands known synonyms or abbreviations.

For Khmer questions, the system includes Unicode normalization, dictionary-based word segmentation, and Romanized-Khmer mapping. These functions help process questions written in Khmer script, English, or a mixture of both languages.

For example, a citizen may use the abbreviation "MPWT" instead of "Ministry of Public Works and Transport." The system can expand the abbreviation to improve the retrieval of relevant government information.

This algorithm supports REQ-2, REQ-3, and REQ-26 while preparing the input for RetrievalService.

2. Hybrid Information Retrieval

SuperAsk.AI uses a hybrid retrieval approach to find relevant government information from its approved knowledge base. As described in Sections 2 and 3, RetrievalService coordinates two search methods through the SearchIndex interface:

- **KeywordIndex (BM25):** Evaluates the relevance of government passages based on matching search terms, term frequency, inverse document frequency, and document length.
- **VectorIndex:** Represents questions and stored passages as numerical vectors and calculates their similarity using cosine similarity. The design supports meaning-based retrieval through embeddings.

The system combines results from both search methods using Reciprocal Rank Fusion (RRF). The design also includes a reranking stage, described in Section 3 as cross-encoder-style reranking, to prioritize relevant passages. The current implementation uses lexical matching signals for reranking, while the architecture allows more advanced reranking methods to be incorporated.

For example, when a user asks, "What documents are required to renew my passport?", the retrieval system searches for government passages related to passport renewal and required documents. The resulting candidates are merged, reranked, and filtered to exclude superseded or not-yet-effective content.

The retrieval process returns the highest-ranked passages with their source information for answer generation.

This algorithm supports REQ-4, REQ-5, and REQ-8.

3. Three-Tier Answer Selection

The QAController uses a three-tier decision process to determine how a question should be answered. This process follows the pipeline defined in Section 3.

- **Tier 1 – Curated Answer:** If the question matches an approved curated answer above the configured threshold, the system returns that answer directly.
- **Tier 2 – Retrieval-Augmented Generation:** If no suitable curated answer is found, RetrievalService retrieves relevant passages, and AIModelGateway uses them to generate a grounded answer.
- **Tier 3 – Refusal: If the question** is unsupported, outside the system's scope, or lacks sufficiently relevant sources, the system declines to provide an unsupported answer and tells the user which areas it covers.

QAController applies policy guardrails and a confidence gate before proceeding with answer generation. The current design uses a curated-answer threshold of 0.72 and a minimum retrieval relevance threshold of 0.20, as specified in Section 2.

This process reduces the risk of generating answers without sufficient supporting information and supports REQ-7 and REQ-22.

4. Answer Verification

After AIModelGateway generates a response, AnswerVerifier checks whether important factual details are supported by the retrieved passages.

The verification process examines information such as fees, dates, deadlines, document requirements, and numerical values. It also checks that referenced citation identifiers correspond to retrieved sources.

If an answer contains an unsupported factual claim, the system can withhold the generated response instead of presenting it as verified.

For example, if an answer provides a passport-processing fee that does not appear in its cited passage, AnswerVerifier identifies the unsupported information.

CitationPresenter then presents the supporting sources separately from the answer text.

This process supports REQ-6, REQ-22, and REQ-28.

5. Official Content Change Detection

For UC-11: RefreshOfficialContent, RefreshController coordinates the monitoring of registered official government sources.

SourceRegistry maintains the list of registered sources, while the SourceCrawler interface defines how content is retrieved from different platforms.

The current design includes WebsiteCrawler, FacebookCrawler, and TelegramCrawler. WebsiteCrawler is implemented, while the Facebook and Telegram crawler adapters remain planned.

The content-refresh process performs the following operations:

1. Load registered official sources from SourceRegistry.
2. Retrieve permitted source content using the appropriate crawler.
3. Extract relevant information and compare it with previously stored content.
4. Identify new, modified, removed, or unreachable content.
5. Classify detected changes and create ChangeSignal records.
6. Place detected changes into ReviewQueue for human review.
7. Update the approved knowledge base only after steward approval.

The design separates automated content collection from approval. The crawler does not directly insert unverified information into ContentStore.

This process supports REQ-29 and REQ-30 by allowing government information and registered sources to be updated without changing the core question-answering logic.

Overall, these algorithms support reliable information retrieval, grounded answer generation, and controlled updates to official government-service content.

b. Data Structures

SuperAsk.AI uses arrays, maps, structured objects, and indexed records to organize government-service information, maintain conversation history, and process retrieval results. These data structures correspond to the design classes and attributes described in Section 2.

1. Arrays and Lists

Arrays and lists store collections of government passages, search candidates, source citations, conversation messages, and registered sources.

For example, ContentStore maintains a list of approved Passage objects, while RetrievalService returns a ranked list of Candidate objects.

Arrays are appropriate because the system frequently processes collections of results in a specific order, particularly when ranking retrieved passages or displaying citations.

2. Maps and Hash-Based Structures

Maps provide efficient lookup using unique identifiers or search terms.

As described in Section 2, KeywordIndex uses a postings structure that maps search terms to passage identifiers and term frequencies. ContentStore also uses a map to retrieve passages by their identifiers.

These structures reduce the need to search through every stored record when locating a particular passage or calculating keyword relevance.

Maps were selected for efficient access and flexibility as the knowledge base expands.

3. Structured Objects and Records

Government information is organized into structured Passage objects containing fields such as ministry, document title, legal instrument, article, source URL, effective date, last-verified date, review date, and content.

A SourceCitation object contains the information needed to display an official source beneath an answer.

Using structured objects allows the system to maintain the relationship between government information and its supporting source metadata.

This organization supports REQ-4, REQ-6, and REQ-28.

4. Retrieval Candidates and Numerical Vectors

The retrieval system uses Candidate objects to store lexical scores, vector similarity scores, fused scores, and reranking results.

VectorIndex uses numerical vectors represented as lists of real numbers, with passage identifiers mapped to their corresponding vectors.

These structures allow RetrievalService to compare the results of different search methods and select the most relevant government passages.

5. Conversation History

ConversationHistory manages Conversation objects containing ordered lists of user and assistant messages.

The web application maintains conversation history in browser storage, while Telegram maintains short per-chat conversation history on the server.

Ordered message lists preserve conversation sequence, and keyed structures allow previous conversations to be identified and retrieved.

This supports UC-6: ViewRecentConversations and UC-8: AskFollowUpQuestion.

Overall, these data structures were selected based on retrieval efficiency, flexibility, maintainability, and compatibility with the TypeScript and Next.js architecture.

c. Concurrency

SuperAsk.AI handles multiple user requests through its web application and Telegram interfaces. The system uses asynchronous request processing rather than assigning manually managed threads to individual design objects.

The web application and Telegram bot can receive requests independently. Each request passes through the API Gateway and Controller subsystem, where QAController coordinates language processing, information retrieval, answer generation, and verification.

External AI service requests are handled asynchronously, allowing the server environment to manage other incoming requests while waiting for responses.

The system also includes a separate content-refresh process controlled by RefreshController. As described in Section 3, this process runs through a scheduled task on a separate machine rather than within the citizen-facing web server.

Because the question-answering and content-refresh processes operate separately, synchronization is important when managing shared government information.

The design addresses this by allowing citizen-facing retrieval to access approved ContentStore records while newly detected changes remain in ReviewQueue until reviewed. The steward-approval workflow is intended to prevent unapproved content from becoming available to users.

Conversation histories are also separated by conversation or chat identifier to prevent messages from different users from being mixed.

No custom thread synchronization mechanism is currently specified for the individual design classes. Concurrency is primarily managed by the Node.js runtime, serverless hosting environment, and the separation of background content-refresh operations.

Further testing will evaluate whether simultaneous requests are processed correctly and whether conversation history and approved content remain consistent.

5. User Interface Design and Implementation

The SuperAsk.AI user interface was initially presented in Report One, Section 4, through preliminary screen mock-ups and user effort estimates. Since then, the team has continued implementing the interface using React and Next.js, following the Presentation Subsystem described in Section 3.

The implemented interface retains the main design established in Report One: a chat-based system where citizens can submit questions, receive government-service information, and view supporting official sources.

The primary improvements focus on connecting the initial interface designs to the actual question-answering system, improving user feedback, and supporting additional access through Telegram. The team has retained the original navigation structure to avoid unnecessary increases in user effort.

a. Chat Interface and Question Submission

In Report One, the initial mock-up allowed users to access the chat interface, enter a government-service question, and submit it through the Send button.

The current implementation maintains this interaction. ChatUI collects the question and sends it through the backend API, where QAController coordinates the question-answering process.

The interface also displays the submitted question and provides a loading indicator while the system processes the request.

This allows users to see that their question has been submitted without navigating away from the conversation.

The main interaction remains simple: the user selects the input field, enters a question, and submits it.

These functions support REQ-21, REQ-31, REQ-32, and REQ-33.

b. Answer Presentation and Verified Sources

Report One introduced a structured answer layout containing important government-procedure information, including required documents, steps, fees, deadlines, and responsible offices.

The current implementation connects the answer interface to the retrieval and answer-generation pipeline described in Sections 2 and 3.

The interface includes components for displaying answers and source citations. CitationPresenter is responsible for presenting source information separately from the main answer text.

This allows users to review the response and identify its supporting government source without conducting an additional search.

For example, a citizen asking about passport renewal can receive the available procedure information together with the corresponding official citation.

The source interface can also display freshness information when a citation may require further verification.

This supports REQ-6, REQ-16 to REQ-19, REQ-28, and REQ-34.

c. Conversation History and Follow-Up Questions

The preliminary interface in Report One included a Recent Conversations section to help users return to previously discussed government-service topics.

The system design includes ConversationHistory, which stores previous exchanges and provides context for follow-up questions.

For example, after asking about driving licence renewal, a citizen may ask, "How much does it cost?" The system uses the earlier conversation to identify the procedure being discussed.

This reduces the need to repeat the original question or reenter the full procedure name.

The web interface stores conversation history on the device, and the Telegram bot keeps the last eight turns of each chat in memory; persistent Telegram history remains part of ongoing development.

These functions support REQ-10, REQ-14, REQ-36, UC-6, and UC-8.

d. Bilingual Interface

SuperAsk.AI supports Khmer and English interaction, consistent with the interface requirements established in Report One.

The user interface provides language selection, while LanguageService processes the language of submitted questions.

The same general navigation and question-submission workflow is maintained in both languages.

This avoids creating separate procedures for Khmer-speaking and English-speaking users.

The interface also needs to display Khmer characters correctly across supported browsers and devices.

This supports REQ-2, REQ-3, REQ-26, REQ-37, and UC-10: SwitchLanguage.

e. Telegram Bot and Mini App Integration

One significant implementation development since Report One is the integration of Telegram access.

The system includes a Telegram bot that receives user questions through the Telegram Bot API and forwards them to the backend question-answering workflow.

Users can also open the SuperAsk.AI chat interface through a Telegram Mini App.

This provides two methods of interaction: submitting text messages directly through Telegram or opening the web-based chat interface inside the application.

Both access methods use the same core question-and-answer functionality described in Section 3.

This reduces the need for users to open a separate website when they are already using Telegram.

Messenger integration remains planned rather than implemented.

This development supports UC-9: AskThroughMessagingChannel and partially fulfills REQ-15.

f. User Effort and Significant Interface Changes

Report One estimated the number of interactions required for common tasks, including submitting a question, selecting a popular procedure, viewing a verified source, opening office information, and asking follow-up questions.

Most of these interactions were already designed to require only a small number of user actions. Therefore, the implementation focuses primarily on preserving the original ease of use while connecting the interface to the functional backend.

The following table summarizes the main changes and their effect on user effort.

Initial Design in Report One	Implementation in Report Two	Effect on User Effort
Chat input and Send button	Connected web chat with backend question processing and loading feedback	Users can submit questions and receive responses within the same interface.
Structured answer mock-up	Answer presentation integrated with retrieved government information	Users can review procedure details without searching multiple websites.
Verified Source section	Dedicated citation components connected to retrieved sources	Users can access source information directly from the answer.
Recent Conversations interface	Conversation-history functionality and follow-up context	Reduces the need to repeat earlier questions.
Khmer and English interface design	Bilingual interface and language-processing integration	Maintains a similar workflow across both languages.
Web-based interface	Additional Telegram bot and Mini App access	Allows users to interact through a familiar messaging application.
Office contact card and “Show office on	The answer card lists the responsible office, and a “Show	Office details appear in the answer, and one click opens the

map” (REQ-38, REQ-39, UC-7)	office on map” link appears under the answer. A fuller contact card (ChatUI.showContactCard with OfficeDirectory, Section 2) is in progress.	map, as estimated in Report One.
“Was this helpful?” Yes/No feedback	Replaced by a “Report an error” link under each answer that opens a short dialog (optional comment, then Submit); ChatUI.reportAnswer sends the user’s report to FeedbackLog (Section 2).	Reporting an error takes two clicks instead of one; it stays optional and adds no steps to asking a question.
Popular Procedures sidebar (REQ-35, UC-5)	Retained. ChatUI.selectPopularProcedure sends the selected procedure as a question.	One click and no typing, as estimated in Report One.
Out-of-scope reply (Report One Figure 11, UC-4)	ChatUI.displayDecline replies that the system only answers questions about Cambodian government procedures, suggests topics such as a licence, certificate, registration or permit (Section 2).	Users get a clear next step instead of a dead end.
Messenger tab (REQ-15)	The design keeps a Messenger tab next to Web and Telegram. Integration is planned, not yet implemented (tentative in the Plan of Work).	No change for web and Telegram users.

The implementation does not significantly change the number of actions required to submit a basic question. Instead, it improves the functionality available after submission, including retrieval, citations, conversation history, and multiple access channels.

The following table compares the user effort estimated in Report One with the current design.

Task	**Report One Estimate**	**Report Two Design**	**Change**
1. Ask a government-service question	Navigation: 1 click. Data entry: 1 click and 34 key presses.	Same steps. A loading indicator now shows that the question is being processed.	No change
2. Select a suggested procedure	1 click	1 click (ChatUI.selectPopularProcedure)	No change
3. View the verified source	1 click	Source card shown under the answer; 1 click opens the official page.	No change
4. View the office location	1 click	“Show office on map” link under the answer (1 click)	No change
5. Ask a follow-up question	Navigation: 1 click. Data entry: 1 click and 22 key presses.	Same steps. Earlier context is reused, so the procedure name does not need to be retyped.	No change
6. Start a new chat	1 click	1 click	No change

7. Ask through Telegram	Not estimated in Report One	Open the bot chat (1 click), type the question (34 key presses for the same example), and send (1 click).	New channel

Further usability testing will determine whether users can complete the main tasks without assistance, as required by REQ-21.

Overall, the interface implementation follows the original design while improving access to verified government-service information and maintaining a straightforward user experience.

6. Design of Tests

The purpose of testing is to verify that SuperAsk.AI meets the functional, nonfunctional, and user-interface requirements established in Report One.

The testing strategy follows the system architecture and class responsibilities described in Sections 2 and 3. Particular attention is given to QAController, LanguageService, RetrievalService, AIModelGateway, AnswerVerifier, CitationPresenter, and RefreshController.

The repository already contains more than 600 automated unit and integration tests, as described in Section 3. This section designs the additional test cases for the first demonstration. The planned tests will be programmed, executed, and evaluated during the demonstration preparation stage.

a. Test Cases

The following test cases describe the inputs, expected behavior, and requirements associated with the major system functions.

Test ID	Related Requirements / Use Cases	Test Input or Scenario	Expected Result
TC-01	REQ-1, REQ-31, REQ-32; UC-1	Submit "How do I renew my driving licence?" through the web chat.	QAController processes the request and returns an answer or an appropriate decline.

TC-02	REQ-2, REQ-26, REQ-37; UC-1, UC-10	Submit a government-service question written entirely in Khmer.	LanguageService identifies Khmer input, and the system displays readable Khmer text.
TC-03	REQ-3; UC-1	Submit a question containing both Khmer and English terms.	The system processes the mixed-language question and retrieves relevant information.
TC-04	REQ-4, REQ-5; UC-2	Submit a question matching a known approved government passage.	RetrievalService returns the relevant passage with its source metadata.
TC-05	REQ-6, REQ-28, REQ-34; UC-3	Request information that has a verified official source.	CitationPresenter displays the citation separately from the answer, including available source details.
TC-06	REQ-7, REQ-23; UC-4	Submit "What is the capital of France?"	The system identifies the request as outside its government-service scope and does not generate an unsupported government-service answer.
TC-07	REQ-14; UC-8	Ask about driving licence renewal, followed by "How long does it take?"	ConversationHistory supplies the earlier context so the follow-up refers to the correct procedure.
TC-08	REQ-9, REQ-35; UC-5	Select a popular procedure from the interface.	The selected procedure is submitted without manually typing the full question.

TC-09	REQ-10, REQ-36; UC-6	Select a previously saved conversation.	The corresponding conversation and messages are displayed.
TC-10	REQ-11, REQ-12, REQ-38; UC-7	Ask for contact and location information for an office with a verified record.	OfficeDirectory returns the available office details, including address and contact information.
TC-11	REQ-13, REQ-39; UC-7	Request information for an office without verified online contact details.	The system indicates that verified contact information is unavailable and provides a verified location if one exists.
TC-12	REQ-15; UC-9	Submit a government-service question through Telegram.	TelegramBot forwards the request through the backend workflow and sends the response to the correct chat.
TC-13	REQ-22; UC-1	Supply a controlled AI response containing a fee or date absent from its cited passage.	AnswerVerifier identifies the unsupported claim and prevents it from being presented as verified.
TC-14	REQ-29, REQ-30; UC-11	Register an additional official source in SourceRegistry.	The source can be added without modifying the core answer-generation logic.
TC-15	REQ-25	Open the chat interface at 360px width and at standard desktop resolution.	The interface remains usable without horizontal scrolling.

TC-16	REQ-27	Inspect browser-accessible resources and client-side network responses.	API keys and other protected credentials are not exposed.
TC-17	REQ-24	Submit a controlled sample of government-service questions and measure response-start times.	At least 90% of requests begin displaying a response within five seconds under normal test conditions.
TC-18	REQ-4, REQ-8, REQ-29; UC-11	Run WebsiteCrawler against controlled content that has been modified since the previous stored version.	RefreshController detects the change and places it in ReviewQueue without directly updating approved content.
TC-19	REQ-7, REQ-22; UC-1	Simulate an unavailable external AI model while relevant passages exist.	AIModelGateway uses its extractive fallback or an appropriate safe response according to the defined pipeline.
TC-20	REQ-4, REQ-5; UC-2	Test retrieval using a known ministry abbreviation and its full name.	Query expansion and retrieval return the expected relevant official passages.
TC-21	REQ-16, REQ-17, REQ-18, REQ-19, REQ-20, REQ-33; UC-1	Ask “How do I renew my driving licence?” through the web chat.	The answer displays the required documents, the steps in order, the official fee and any late fee, the validity period, and office hours, where the official source provides them.

TC-22	REQ-21; UC-1	Give test users the chat interface without instructions and ask them to submit a government-service question.	At least 90% of test users submit a question without assistance.
TC-23	REQ-7; UC-4	Submit a government-service question that no approved source covers.	The decline message lists the areas SuperAsk.AI covers, no answer is generated, and the refusal is recorded by FeedbackLog.

These cases cover normal system behavior, unsupported requests, external service failures, and the main integration points described in the class diagrams and system architecture.

Features that are still under development will be tested once their implementations are ready.

Unit Test Cases

The test cases above check the system as a whole. The following unit test cases check individual classes and operations from Section 2 with fixed inputs, so that each part can be tested on its own before integration. UT-02, UT-04, UT-05, UT-07, UT-08, UT-09, UT-10 and UT-14 already have equivalent tests in the repository and run in CI; UT-01, UT-03, UT-11, UT-12 and UT-13 are new, and UT-12 depends on the OfficeDirectory class that is still being implemented.

Test ID	**Class and Operation**	**Input**	**Expected Result**
UT-01	LanguageService.detectLanguage	A Khmer question, an English question, and a mixed question	Returns Khmer, English, and Khmer (any Khmer script selects Khmer) respectively

UT-02	LanguageService.romanisedToKhmer	A Romanised Khmer phrase	Returns the correct Khmer script
UT-03	KeywordIndex.search	A query containing a term found in one known passage	That passage is ranked first
UT-04	VectorIndex.cosine	The same text twice, then two unrelated texts	Score of 1.0, then a score near 0
UT-05	RetrievalService.mergeAndRerank	Two ranked lists that share one passage	The shared passage is ranked first
UT-06	ContentStore.filterCurrentlyEffective	A superseded passage and a passage that is not yet effective	Both passages are removed
UT-07	QAController.matchCurated	Curated match scores of 0.71 and 0.72	0.71 is not used; 0.72 returns the curated answer
UT-08	QAController.confidenceGate	Top retrieval scores of 0.19 and 0.20	0.19 is refused; 0.20 continues to answer generation
UT-09	AnswerVerifier.verifyClaims	An answer containing a fee that is not in its cited passage	The claim is flagged as unsupported

UT-10	CitationPresenter.markPossiblyOutdated	A citation whose last-verified date is past its review date	The citation is marked as possibly outdated
UT-11	ConversationHistory.getRecentContext	A conversation with 10 earlier turns	Only the last 8 turns are returned (maxTurns = 8)
UT-12	OfficeDirectory.findResponsibleOffice	A known procedure, then an unknown procedure	The correct office, then no office
UT-13	ReviewQueue.queueChangeForReview	A detected content change	The change is stored as pending and is not published
UT-14	FeedbackLog.redact	A user report containing a phone number	The phone number is removed before logging

b. Test Coverage

Testing will cover the system's main functional and nonfunctional requirements, with priority given to the five use cases identified in Report One: UC-1, UC-2, UC-3, UC-7, and UC-11. The current automated suite (24 files, 643 tests) measures 89.6% line and statement coverage, 94.3% branch coverage and 96.0% function coverage, and the build fails below floors of 85%, 85%, 90% and 90%. The crawler, source registry, Telegram channel and interface constants are excluded from that measurement, which is why UC-9 and UC-11 rely on the integration tests below; the suite runs without a model credential, so the AI-model path is always exercised through the extractive fallback.

Functional Testing

Functional testing will verify question submission, language processing, official-source retrieval, answer generation, citation display, conversation history, office information, Telegram interactions, and content refresh.

Tests will cover both successful operations and alternate scenarios, including missing sources and unavailable external services.

Nonfunctional Testing

Nonfunctional testing will evaluate the acceptance criteria established in Report One:

- **REQ-21 – Usability:** At least 90% of test users should submit a question without assistance.
- **REQ-22 – Answer Accuracy:** No sourced answer in the acceptance-test set should contain factual claims unsupported by its displayed source.
- **REQ-23 – Unsupported Questions:** At least 95% of unsupported or out-of-scope questions in the test set should be correctly refused.
- **REQ-24 – Performance:** At least 90% of test queries should begin displaying a response within five seconds under normal conditions.
- **REQ-25 – Responsive Design:** The web interface should remain usable at approximately 360px width through desktop resolutions.
- **REQ-26 – Language Compatibility:** Khmer and English text should display correctly without missing or unreadable characters.
- **REQ-27 – Security:** API keys and protected credentials must remain server-side.

Boundary and Error Testing

Additional tests will examine empty submissions, unsupported questions, missing source information, expired or superseded content, invalid citations, and external AI service failures.

For example, when relevant government content cannot be found, the system should follow UC-4: DeclineUnsupportedQuestion rather than generate an answer without supporting evidence.

c. Integration Testing Strategy

Integration testing will verify that the subsystems described in Section 3 work correctly together.

The testing strategy will follow the flow of information between the major system components.

The team will use a bottom-up integration strategy. Lower-level classes (LanguageService, KeywordIndex, VectorIndex, ContentStore, and AnswerVerifier) are tested first using test drivers. They are then combined into RetrievalService and QAController, and finally connected

to ChatUI and TelegramBot. External services, including the AI model and the Telegram platform, are replaced with stubs until the final end-to-end tests.

1. Presentation and API Gateway Integration

Tests will confirm that questions submitted through ChatUI reach the correct API endpoint and that the response is returned to the appropriate conversation.

2. Language Processing and Retrieval Integration

Tests will verify that LanguageService correctly prepares questions before RetrievalService performs keyword and vector searches.

3. Retrieval and Model Integration

Tests will confirm that RetrievalService returns relevant passages and that AIModelGateway uses the provided sources when generating grounded responses.

4. Verification and Citation Integration

Tests will verify that AnswerVerifier checks generated claims against retrieved passages and that CitationPresenter displays citations corresponding to the answer.

5. Telegram and Backend Integration

Tests will confirm that TelegramBot receives messages through the Telegram webhook, passes them to the backend, and returns the correct response to the user.

6. Content Refresh Integration

Tests will verify that RefreshController retrieves registered sources through SourceCrawler, detects changes, and places them into ReviewQueue.

The integration tests will also confirm that detected changes do not automatically overwrite approved ContentStore information.

7. Feedback and Audit Integration

Tests will confirm that a user report reaches the /api/feedback endpoint and is recorded by FeedbackLog, with personal data removed before logging.

Where necessary, the team will use controlled inputs and mock external-service responses to isolate components before conducting complete end-to-end testing.

d. Testing of Algorithms, Nonfunctional Requirements, and User Interface

Retrieval Algorithm Testing

The team will prepare questions with known expected sources to evaluate BM25, vector similarity, reciprocal-rank fusion, and reranking.

The tests will include Khmer and English questions, ministry abbreviations, alternative wording, and follow-up questions.

Retrieved results will be compared against known relevant passages to identify ranking and retrieval problems.

Answer Verification Testing

Controlled answers containing supported and unsupported factual claims will be passed through AnswerVerifier.

The team will verify whether unsupported fees, dates, numerical values, and invalid citation references are correctly detected.

User Interface Testing

Participants will be asked to complete the tasks identified in Report One's user effort estimation, including submitting a question, selecting a popular procedure, opening a source citation, viewing office information, asking a follow-up question, and starting a new conversation.

The team will record successful and unsuccessful task completion, the number of user interactions, and any difficulties encountered.

Testing will also include Khmer and English text display on mobile and desktop screen sizes.

Content Refresh Testing

The team will test WebsiteCrawler and RefreshController with controlled examples of unchanged content, updated content, inaccessible sources, and blocked pages.

The purpose is to verify the detection and review process before content enters the approved knowledge base.

Concurrency and Performance Testing

The team will submit multiple requests using separate conversation identifiers to check that responses and histories remain independent.

Response-start times will also be measured to evaluate the performance requirement stated in REQ-24.

e. Testing Schedule and Evaluation

The testing activities will follow the project schedule presented in Section 7: Project Management.

Before the first demonstration on November 9, 2026, the team will implement and run the designed tests, focusing on the core question-answering process, official-source retrieval, citations, guardrails, and integration between the major subsystems.

Each executed test will be documented with its test identifier, input, expected result, actual result, and pass/fail status.

Any failed tests will be reviewed and corrected before the relevant functionality is demonstrated.

Following the first demonstration, the team will incorporate feedback, expand testing coverage, and evaluate improvements to retrieval accuracy, bilingual responses, and usability.

The final testing phase will focus on confirming consistency between the implemented system, design documentation, and requirements before the final demonstration on December 4, 2026.

Project Management

a. Project Coordination and Progress Report

Implemented Use Cases

1. Citizen Q&A – Web

Users can access the SuperAsk.AI web application and submit questions about Cambodian government services in Khmer or English. The system retrieves information from its knowledge base and provides answers with relevant source citations.

2. Citizen Q&A – Telegram

Users can submit government-service questions through the Telegram bot and receive responses directly in the Telegram conversation.

3. Telegram Mini App

Users can access the SuperAsk.AI web chat interface through Telegram's Mini App or interact with the system through regular Telegram messages.

4. Bilingual Support

The landing page and chat interface support English and Khmer. The system also processes questions based on the language used by the user.

5. Grounded Answers and Declines

SuperAsk.AI retrieves government-service information from its approved knowledge base and provides source citations. When sufficient verified information is unavailable, the system can decline to provide an unsupported answer.

Currently Functional

- Landing page and web chat interface
- Question-and-answer API and information retrieval pipeline
- Telegram bot text responses
- Telegram Mini App access
- English and Khmer interface support
- Source citation display
- Basic feedback functions
- Production deployment and health checks

Currently Being Worked On

- Real user authentication
- Retrieval accuracy and AI response improvements
- Expansion and verification of the government-service knowledge base
- Feedback-report handling improvements
- Content management and access-control functions
- Telegram conversation history and multi-turn interaction improvements
- Office contact and location lookup (UC-7)
- Content refresh and change detection with steward review (UC-11)

Other Project Coordination Activities

As of October 21, 2026, the team has completed Report Two, Part One and Part Two, covering the interaction diagrams, class diagrams, interface specifications, and system architecture. For the integrated Full Report Two submission, the team has also prepared Sections 4–6, covering algorithms and data structures, user interface design and implementation, and the design of tests.

In addition to documentation, development continues on authentication, retrieval improvements, content management, Telegram functions, and the remaining use cases. The web application,

Telegram bot, and Mini App use the same core question-and-answer system, while more advanced functions are still under development.

The team is reviewing the consistency between Report One, Report Two, the current implementation, and the planned testing activities. Particular attention is being given to ensuring that the system architecture, design diagrams, and described functionality accurately reflect the available prototype.

The next development phase will focus on completing the remaining functions, improving bilingual response accuracy, conducting integration testing, and preparing for the first demonstration on November 9, 2026. Following the demonstration, the team will use feedback and testing results to guide further improvements before the final demonstration on December 4, 2026.

b. Plan of Work

As of October 21, 2026, the team has completed the documentation for Report Two, Parts One and Two, and prepared the remaining sections required for the integrated Full Report Two submission. The prototype already supports web-based question answering, Telegram access, bilingual interaction, official-source citations, and basic feedback.

The current development activities focus on authentication, information retrieval improvements, content management, access control, and Telegram conversation history. The team is also preparing the remaining use cases, particularly UC-7: GetOfficeContactAndLocation and UC-11: RefreshOfficialContent.

After the Full Report Two submission, the team will prioritize completing these functions and testing their integration with the existing system. The first demonstration is scheduled for November 9, 2026. The remaining project period will focus on addressing feedback, improving accuracy and usability, expanding verified government information, and completing the system for the final demonstration on December 4, 2026.

Plan of Work – Report Two to Final Demonstration

Period	Planned Work	Milestone
Sep 21–30 (Completed)	Completed requirements, business goals, use cases, preliminary interface designs, and project management documentation. Reviewed and integrated Report One.	Full Report #1 – Sep 30

Oct 1–6 (Completed)	Prepared interaction diagrams for the selected use cases. Continued development of web chat, retrieval, citations, bilingual functions, Telegram integration, and the deployed prototype.	Report #2 Part 1 – Oct 6
Oct 7–12 (Completed)	Prepared class diagrams, interface specifications, traceability matrices, and system architecture documentation. Continued improvements to authentication, retrieval, Telegram history, content management, and access control.	Report #2 Part 2 – Oct 12
Oct 13–21 (Current)	Integrated and reviewed Report Two, including Sections 4–6 and Project Management. Continued authentication, content management, retrieval improvements, and planning for UC-7 and UC-11. Prepared integration test designs.	Full Report #2 – Oct 21
Oct 22–Nov 1	Continue implementing UC-7 and UC-11 and integrate remaining functions. Test Khmer and English responses against approved official sources.	First-demo implementation
Nov 2–8	Debug the system, test unsupported and out-of-scope questions, evaluate response time and usability, and prepare the demonstration.	First-demo preparation
Nov 9	Demonstrate the first working iteration and collect feedback from the professor.	Demo #1
Nov 10–16	Apply feedback from Demo #1, improve the interface, expand verified government information and ministry coverage, and consider Messenger integration if confirmed.	Report #3 Part 1 – Nov 16
Nov 17–23	Complete system testing and final documentation. Verify consistency between requirements, implementation, use cases, and reports.	Full Report #3 – Nov 23
Nov 24–Dec 3	Resolve remaining defects, refine the retrieval system, finalize supported government-service coverage, and prepare the final demonstration.	Final refinements

Dec 4	Present the completed SuperAsk.AI system and demonstrate its final functionality.	Demo #2

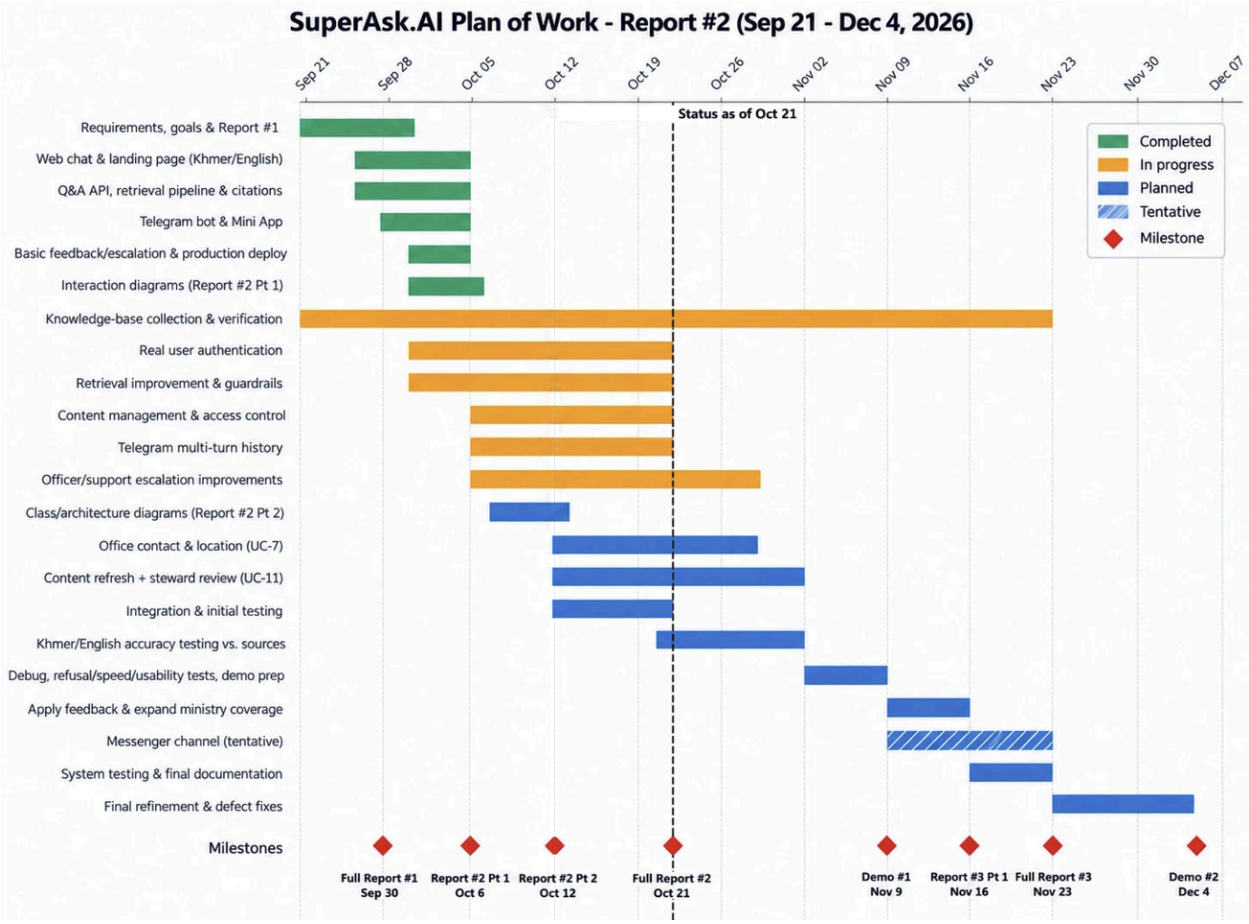


Figure 13: SuperAsk.AI Project Gantt Chart – Updated as of October 21, 2026.

c. Team Responsibilities

The following table summarizes each team member's completed work, current responsibilities as of October 21, 2026, and planned contributions for the remaining development period. Responsibilities are divided according to each member's project role, while integration, testing, and documentation require coordination across the team.

Team Member	Role	Work Completed / So Far	Current Responsibilities	Future Responsibilities
Yean Fong	Project Lead	Coordinated requirements, project scheduling, documentation, and team organization. Prepared the Project Coordination and Progress Report, reviewed report consistency, and coordinated team contributions.	Coordinate the Full Report Two submission, update project management and the Gantt chart, review Sections 1–6 for consistency, and assist team members where needed.	Monitor development progress, coordinate Demo #1 preparation, track milestones, and oversee report integration and final project documentation.
Phallamonunhh Chun	AI/ML Engineer and System Design	Developed the RAG pipeline, retrieval functions, AI model integration, and guardrails. Prepared interaction diagrams, class diagrams, interface specifications, traceability matrices, and revisions to system architecture documentation.	Improve retrieval accuracy, evaluate AI responses, and maintain consistency between system design documentation and the implemented code. Support the technical descriptions of algorithms and data structures.	Refine the retrieval and verification pipeline, support integration testing, improve AI response reliability, and update design documentation when the implementation changes.

Bunkheang Heng	Web/API Developer	Developed the web application, APIs, backend integration, and database-related functions. Contributed to system architecture documentation and Telegram integration.	Continue development of authentication, content management, access control, Telegram conversation history, and other backend improvements.	Support UC-7 and UC-11 implementation, complete backend integration, resolve technical issues, and assist with testing before the demonstrations.
Doungkhe Neang	Data/Content Curator	Conducted government-servic e research, organized content, verified sources, and supported project documentation and coordination.	Continue collecting and organizing verified government-service information. Support knowledge-base expansion and coordinate content requirements for remaining functions.	Prepare additional verified content, support office-contact information collection, and review content accuracy during testing and refinement.
Sereyprophea Sarin	QA/Testing Lead	Contributed to requirement reviews, project documentation, QA planning, and initial testing activities. Supported the preparation and updating of the project schedule.	Support the design of test cases, review functional and nonfunctional requirement coverage, and identify potential usability and system issues.	Coordinate test execution, record results, verify fixes, evaluate usability and response accuracy, and support testing before Demo #1 and Demo #2.

Heejae Choi	UX/UI Designer	Developed the prototype interface, user flows, responsive layouts, and initial UI specifications. Contributed to user-effort evaluation and system architecture documentation.	Review the implemented interface against Report One's designs and user-interface requirements. Refine layouts and interactions where necessary.	Improve usability based on feedback, support interfaces for new features, and refine responsive and bilingual interface behavior before the final demonstration.

The team will continue reviewing progress and coordinating responsibilities as implementation advances. Changes to the system design, requirements, or planned functionality will be reflected in the corresponding documentation to maintain consistency between the reports and the final SuperAsk.AI implementation.

References

SuperAsk.AI Team AO3. (2026). ***SuperAsk.AI GitHub repository*****.**
https://github.com/Monunhh/Super-ask.git
Used for project source code, version control, and team development.

Ministry of Education, Youth and Sport. (2026). ***Ministry of Education, Youth and Sport – Cambodia*****.**
https://moeys.gov.kh/en
Used as an official government source for information related to education services, procedures, requirements, and ministry information used by SuperAsk.AI.

National Election Committee. (2026). ***National Election Committee of Cambodia*****.**
https://www.nec.gov.kh/english/
Used as an official source for voter registration, voter lists, election procedures, polling stations, and other electoral information.

Marsic, I. (2012). ***Software Engineering*****. Rutgers University.**
https://eceweb1.rutgers.edu/~marsic/books/SE/book-SE_marsic.pdf
Used as a reference for software engineering concepts, interaction diagrams, design principles, and assigning responsibilities among system objects.